

Hubei University

Academic Year 2025 -- 2026

Semester 2

Student Laboratory Report Book

Institute: Manchester United College

Student Name: Qin Tian

Class: 2402

Student ID: 202431123002054

Course Course: Big Data Analysis

Instructor: Li Jie

Student Laboratory Rules

1. Students must conduct experiments at the scheduled time and may not be absent or late without cause.

2. Before each experiment, students must preview the assigned experiment and prepare a preview report as required.

3. Before each experiment, students must submit the previous experiment report and the preview report for the current experiment. The experiment may begin only after the instructor has checked and approved them.

4. After reaching their assigned lab station, students must check the instruments they use against the equipment list for any missing or damaged items. Any problems must be reported to the instructor promptly. Using another group's instruments without authorization is strictly forbidden.

5. During the experiment, students must maintain a rigorous scientific attitude and strictly follow instrument operating procedures and precautions.

6. After the experiment, students must submit their data to the instructor for inspection. Once approved, they should restore and

organize the instruments and equipment before leaving the lab.

7. Keep the laboratory quiet and tidy. Loud talking, littering, and eating are prohibited.

8. If an accident or damage to instruments or equipment is caused by violating operating procedures or conducting experiments without permission, it must be reported promptly. A damage list must be completed, and compensation shall be made according to the college's relevant regulations.

Experiment Title:

Lab3-Experiment 4 Clustering Model Practice

Laboratory: Innovation & Entrepreneurship 7001

Date: 2026.5.29

1. Problem Description

This experiment is **Experiment 4: Clustering Model Practice** in the continuous project of **Big Data Analysis and Application**. Different from Experiment 3, this experiment is not a supervised classification task. The main purpose of this experiment is to practise **unsupervised clustering** on the given **wine dataset**. Although the original **wine.data** file is a classification dataset and its first column contains the **class label**, the requirement of this experiment is to remove the label before clustering. Therefore, the model should not learn from the known class. Instead, it should discover possible groups only from the chemical features of the wine samples.

The main task of this experiment is to build clustering models for the wine data and compare their clustering results. In the dataset, the first column is **class**, while the remaining columns are **13 chemical features**, such as **Alcohol**, **Malic acid**, **Ash**, **Magnesium**, **Flavanoids**, **Color intensity**, **Hue**, and **Proline**. According to the requirement of this experiment, the **class** column can only be saved as optional reference information, but it must not be used in model training, parameter selection, or the main evaluation process. The real input of the clustering models is only the feature matrix after dropping the **class** column.

According to **Teacher Li Jie's PPT and class explanation**, this experiment should cover three important types of clustering methods: **Partition-based Clustering**, **Density-Based Clustering**, and **Hierarchical Clustering**. In my implementation, **KMeans** is used as the partition-based clustering model, **DBSCAN** is used as the density-based clustering model, and **AgglomerativeClustering** is used as the hierarchical clustering model. These three models represent different ideas of grouping data. **KMeans** divides samples into a fixed number of clusters according to distance. **DBSCAN** tries to find dense regions and can mark some samples as noise.

AgglomerativeClustering builds clusters step by step by merging similar samples. By comparing these models, I can better understand that clustering is not only about producing labels, but also about understanding different assumptions behind different clustering methods.

One important problem in this experiment is the difference between **classification** and **clustering**. In a classification task, the target label is used during training, and the model learns how to predict the known class. However, in a clustering task, the model should work without using the known label. This means that even though the wine dataset originally has class labels, these labels should be removed before clustering. At first, this point looked like a small data-processing detail, but after thinking about it more carefully, I found that it is actually the key point of the whole experiment. If the **class** label is accidentally included in the feature matrix, the experiment will no longer be a real clustering experiment.

Another important problem is how to evaluate the clustering results correctly. Since this experiment is a clustering task, I should not use classification metrics as the main evaluation criteria. Metrics such as **accuracy**, **precision**, **recall**, **F1-score**, **ROC-AUC**, **PR-AUC**, **confusion matrix**, and **classification report** are suitable for supervised classification tasks, but they are not suitable as the main metrics for this clustering experiment. Instead, the main evaluation metrics should be **Silhouette Coefficient**, **Calinski-Harabasz Index**, and **Davies-Bouldin Index**. These metrics evaluate the compactness and separation of clusters without depending on the original class labels.

In this experiment, the original class label is only used after clustering as an **optional external comparison**. For example, metrics such as **ARI**, **NMI**, **homogeneity**, **completeness**, **v-measure**, and the class-cluster crosstab can help me observe whether the clustering result is similar to the original wine classes. However, these results are only optional references. They cannot be treated as the main evaluation of the clustering models, because the core requirement of the experiment is to complete an unsupervised clustering process after removing the classification label.

At first, this experiment looked easier than the previous classification experiment, because there was no target variable to predict and no train-test split for supervised evaluation. However, after preparing the code and checking the result files, I realized that the real difficulty was not simply running **KMeans**, **DBSCAN**, or **AgglomerativeClustering**. The more important difficulty was keeping the experimental logic correct. I needed to make sure that the **class** label was removed,

that the feature matrix only contained the **13 wine chemical features**, and that the evaluation metrics matched the nature of clustering. This also made me understand more clearly why Teacher Li Jie emphasized the logic behind the model instead of only asking us to run sklearn code.

Teacher Li Jie's class has benefited me a lot in this part. Her explanation helped me notice that a machine learning experiment is not only about whether the code can run successfully. It is also about whether the model, the data, and the evaluation method match the real task. For this experiment, if I used classification metrics as the main result, or if I kept the original label in the feature matrix, the code might still run, but the experiment would be wrong in principle. This reminder made me more careful when designing the workflow and writing the report.

So the main problem in this experiment is how to complete a correct and complete **clustering workflow** on the wine dataset. More specifically, I need to remove the original **class** label, standardize the **13 chemical features**, build and compare **KMeans**, **DBSCAN**, and **AgglomerativeClustering** models, evaluate the clustering results with proper internal clustering metrics, and use result tables and figures to explain the final cluster structures. In my understanding, this experiment is not only a practice of clustering algorithms, but also a practice of distinguishing **supervised learning** from **unsupervised learning** and choosing evaluation methods according to the real task type.

2. Basic Approach

My basic idea in this experiment was to complete the clustering task step by step instead of directly applying one clustering algorithm to the data. Since this experiment is **Clustering Model Practice**, the first thing I needed to do was to make sure that the workflow was really an **unsupervised learning** workflow. The wine dataset originally contains a **class** label in the first column, but this label should not be used in clustering. Therefore, before building any model, I first loaded the **wine.data** file, set clear column names for all variables, saved the first column only as **y_true_optional**, and then removed the **class** column from the feature matrix.

After this step, the actual input data for clustering only contained the **13 chemical features** of wine. These features included **Alcohol**, **Malic acid**, **Ash**, **Alcalinity of ash**, **Magnesium**, **Total phenols**, **Flavanoids**, **Nonflavanoid phenols**, **Proanthocyanins**, **Color intensity**, **Hue**, **OD280/OD315 of diluted wines**, and

Proline. I also checked the basic information of the dataset, including the data shape, missing values, duplicate rows, data types, and feature descriptive statistics. This inspection step was important because clustering models are sensitive to the quality and scale of input features. If I skipped this step, I might not notice whether the input data contained abnormal values or whether some features had much larger ranges than others.

Next, I standardized the feature matrix with **StandardScaler**. This step was necessary because most clustering methods in this experiment are based on distance or similarity. In the wine dataset, different chemical features have very different value ranges. For example, some features have small decimal values, while **Proline** has much larger numerical values. If I used the raw data directly, features with larger scales might dominate the distance calculation, and the clustering result would become biased. After standardization, all 13 features were placed on a comparable scale, which made the later clustering models more reasonable.

I also used **PCA** to reduce the standardized features to two dimensions for visualization. PCA was not used as the main training feature space. It was mainly used to draw clear scatter plots so that I could observe the cluster structure more intuitively. This helped me compare different clustering results visually, especially for **KMeans**, **AgglomerativeClustering**, and **DBSCAN**. However, I understood that a two-dimensional PCA plot is only a visualization tool. The main model training and metric evaluation were still based on the standardized 13-dimensional feature space.

For the first model type, I used **KMeans** as the **Partition-based Clustering** method. In this part, I tested different numbers of clusters from $k = 2$ to $k = 10$. For each value of k , I recorded **inertia**, **Silhouette Coefficient**, **Calinski-Harabasz Index**, **Davies-Bouldin Index**, training time, and cluster size information. I also drew the **elbow curve**, **silhouette curve**, **Calinski-Harabasz curve**, and **Davies-Bouldin curve** to help choose a reasonable number of clusters. Since the original wine dataset is known to have three wine types, but the class label could not be used in clustering, I treated **KMeans with $k = 3$** as an important model for explanation, while still comparing it with the best KMeans setting selected by internal clustering metrics.

For the second model type, I used **DBSCAN** as the **Density-Based Clustering** method. Different from KMeans, DBSCAN does not require the number of clusters to be given directly. Instead, it depends mainly on **eps** and **min_samples**. Therefore, I performed a parameter search over several eps values and min_samples values. For

each parameter setting, I recorded the number of valid clusters, the number of noise samples, the **noise ratio**, and the three internal clustering metrics. I also drew the **k-distance curve**, **DBSCAN silhouette heatmap**, **noise ratio heatmap**, and **cluster count heatmap**. This part helped me understand that DBSCAN can find dense regions and mark uncertain samples as noise, but it is also very sensitive to parameter selection.

For the third model type, I used **AgglomerativeClustering** as the **Hierarchical Clustering** method. I tested several linkage methods, including **ward**, **complete**, **average**, and **single**, and for each linkage method I tested cluster numbers from **2** to **10**. For each setting, I calculated **Silhouette Coefficient**, **Calinski-Harabasz Index**, and **Davies-Bouldin Index**. I also generated comparison figures for the different linkage methods and drew a **dendrogram** to show the hierarchical merging process. This model was useful because it gave me another way to understand clustering: instead of directly assigning samples into fixed groups, it builds a tree-like structure by gradually merging similar samples.

After building the three types of clustering models, I compared them using only proper **internal clustering metrics**. The main evaluation metrics were **Silhouette Coefficient**, **Calinski-Harabasz Index**, and **Davies-Bouldin Index**. A higher Silhouette value usually means that samples are closer to their own cluster and farther from other clusters. A higher Calinski-Harabasz value usually means better cluster separation. A lower Davies-Bouldin value usually means better clustering quality. These metrics are suitable for this experiment because they do not require the original class label.

At the same time, I paid special attention not to use classification metrics as the main evaluation criteria. Metrics such as **accuracy**, **precision**, **recall**, **F1-score**, **ROC-AUC**, **PR-AUC**, **confusion matrix**, and **classification report** were used in the previous classification experiment, but they are not suitable as the main evaluation metrics for this clustering experiment. This was one of the most important differences between Experiment 3 and Experiment 4. In Experiment 3, the target label was used for supervised learning and validation evaluation. In this experiment, the **class** label had to be removed before clustering, so the evaluation method also had to change.

Although the original wine class label was removed from the clustering process, I still kept it as **optional external comparison** after the models had already produced clustering results. This means that metrics such as **ARI**, **NMI**, **homogeneity**, **completeness**, **v-measure**, and the class-cluster crosstab were only used as

additional reference information. They helped me observe whether the discovered clusters were similar to the original wine classes, but they were not used for training, parameter selection, or main model evaluation. I think this design made the experiment more complete while still following the requirement that the classification labels should be removed before clustering.

After the model comparison, I saved the numerical results into the **results** folder. The result files included basic data information, feature descriptive statistics, KMeans selection results, AgglomerativeClustering grid results, DBSCAN grid search results, final model comparison, model ranking summary, cluster profile summary, final cluster assignments, and optional external label comparison files. I also made sure that **final_cluster_assignments.csv** did not contain the original **class** label. It only stored the sample id and cluster labels produced by different clustering models. This was important because the final clustering output should not mix the true class label into the main result file.

At the same time, I generated figures in the **figures** folder. These figures included feature boxplots before and after scaling, feature correlation heatmap, PCA visualization, KMeans elbow and metric curves, AgglomerativeClustering comparison plots, the hierarchical dendrogram, DBSCAN parameter heatmaps, final model metrics comparison, cluster size comparison, and cluster profile heatmap. These visual results made the experiment easier to explain. For example, the KMeans elbow curve and silhouette curve helped me discuss the choice of cluster number, the DBSCAN heatmaps helped me explain the sensitivity of density-based clustering, and the cluster profile heatmap helped me describe the characteristics of different clusters.

Finally, I used the final clustering results to analyze the cluster profiles. For the main interpretation model, I focused more on **KMeans with $k = 3$** , because it assigns every sample to a cluster and gives a clear three-cluster structure. Although **DBSCAN** achieved strong internal metric values under one parameter setting, it also produced a relatively high noise ratio, so it was more suitable as a density-based comparison model rather than the main model for explaining all wine samples. This comparison helped me understand that the model with the best metric value is not always the easiest or most stable model to interpret in a report.

Overall, my approach was to keep the experiment correct, complete, and reproducible. I first removed the original **class** label, then standardized the **13 wine chemical features**, built three types of clustering models, evaluated them with internal clustering metrics, saved enough result files and figures, and finally used

cluster profiles to explain the result. Compared with the previous classification experiment, this experiment made me more careful about the relationship between the task type, the input data, and the evaluation metrics. For me, the most important idea in this approach was not just to run clustering algorithms, but to make sure that the whole workflow really matched the meaning of **clustering**.

3. Source Code

```
1 from pathlib import Path
2 import sys
3 import pandas as pd
4
5 # Resolve the Experiment 4 folder from either the repo root or this notebook folder.
6 current_dir = Path.cwd().resolve()
7 experiment4_name = "\u5b9e\u9a8c\u5b9e\u9a8c"
8 if (current_dir / "experiment4_wine_clustering.py").exists():
9     exp4_dir = current_dir
10 elif (current_dir / experiment4_name / "experiment4_wine_clustering.py").exists():
11     exp4_dir = current_dir / experiment4_name
12 else:
13     exp4_dir = Path(__file__).resolve().parent if "__file__" in globals() else current_dir
14
15 if str(exp4_dir) not in sys.path:
16     sys.path.insert(0, str(exp4_dir))
17
18 from experiment4_wine_clustering import (
19     resolve_paths,
20     ensure_output_dirs,
21     prepare_data,
22     run_kmeans_experiment,
23     run_agglomerative_experiment,
24     run_dbSCAN_experiment,
25     run_final_analysis,
26 )
27
28 # All generated tables and figures are written under these resolved paths.
29 paths = resolve_paths(exp4_dir)
30 ensure_output_dirs(paths)
31 npaths
32
33 ({'project_root': WindowsPath('D:/daima/cursor/大数据分析'),
34   'exp4_dir': WindowsPath('D:/daima/cursor/大数据分析/experiment4'),
35   'data_dir': WindowsPath('D:/daima/cursor/大数据分析/experiment4/data'),
36   'output_dir': WindowsPath('D:/daima/cursor/大数据分析/experiment4/output'),
37   'figures_dir': WindowsPath('D:/daima/cursor/大数据分析/experiment4/figures')})
```

```
1 # prepare_data reads wine.data, removes the label from X, scales features, and saves checks.
2 data_outputs = prepare_data(paths)
3 data = data_outputs["data"]
4
5 df = data["df"]
6 X = data["X"]
7 y_true_optional = data["y_true_optional"]
8
9 df.head()
```

```
1 # The clustering feature matrix must not contain the original class label.
2 print(f"class in X columns: {'class' in X.columns}")
3 print(f"X shape: {X.shape}")
4 print(f"y_true_optional shape: {y_true_optional.shape}")
5 X.head()
```

```

1 # These generated CSV files document basic data quality and feature statistics.
2 display(pd.read_csv(paths["results_dir"] / "data_basic_info.csv").head(20))
3 display(pd.read_csv(paths["results_dir"] / "missing_duplicate_check.csv").head(20))
4 display(pd.read_csv(paths["results_dir"] / "feature_descriptive_statistics.csv").head())

```

✓ 0.1s

```

1 # All clustering models use standardized features because the wine variables have different scales.
2 X_scaled = data["X_scaled"]
3 X_scaled.describe().T.head()

```

5] ✓ 0.0s

```

1 # PCA is used only for two-dimensional visualization, not for model selection.
2 X_pca_2d = data["X_pca_2d"]
3 pca = data["pca"]
4 print("Explained variance ratio:", pca.explained_variance_ratio_)
5 X_pca_2d.head()

```

[6] ✓ 0.0s

```

1 # KMeans records inertia for the elbow curve and internal metrics for evaluation.
2 kmeans_outputs = run_kmeans_experiment(data, paths)
3 kmeans_outputs["results"]

```

[7] ✓ 22.3s

```

1 # Hierarchical clustering compares linkage strategies on the same scaled features.
2 agglomerative_outputs = run_agglomerative_experiment(data, paths)
3 agglomerative_outputs["results"].head()

```

[8] ✓ 4.1s

```

1 # DBSCAN grid search keeps invalid settings as NaN when too few clusters are found.
2 dbscan_outputs = run_dbscan_experiment(data, paths)
3 dbscan_outputs["results"].head()

```

[9] ✓ 6.4s

```

1 # Final ranking uses internal clustering metrics and checks DBSCAN noise ratio.
2 final_outputs = run_final_analysis(
3     data,
4     kmeans_outputs,
5     agglomerative_outputs,
6     dbscan_outputs,
7     paths,
8 )
9 final_outputs["final_model_comparison"]

```

[10] ✓ 4.7s

```

1 # KMeans k=3 is profiled in original and standardized feature spaces.
2 display(final_outputs["cluster_profile_summary"])
3 display(final_outputs["cluster_profile_scaled_mean"])
4 print(final_outputs["cluster_profile_interpretation"])

```

[11] ✓ 0.0s

```
1 # These external metrics use labels only after clustering and are optional.
2 optional_external = pd.read_csv(paths["results_dir"] / "optional_external_label_comparison.csv")
3 optional_external

[12] ✓ 0.0s
```

```
1 # The recommendation text is based on the final internal-metric comparison table.
2 print(final_outputs["analysis_text"])
3 print("Generated results directory:", paths["results_dir"])
4 print("Generated figures directory:", paths["figures_dir"])

[13] ✓ 0.0s
```

```
from __future__ import annotations

import shutil
import time
import warnings
import zipfile
from pathlib import Path
from typing import Any

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.cluster import AgglomerativeClustering, DBSCAN, KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (
    adjusted_rand_score,
    calinski_harabasz_score,
    completeness_score,
    davies_bouldin_score,
    homogeneity_score,
    normalized_mutual_info_score,
    silhouette_score,
    v_measure_score,
)
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42
FIGURE_DPI = 300
```



```
# Keep the dataset schema explicit so the label column can be separated
safely.
WINE_COLUMNS = [
    "class",
    "Alcohol",
    "Malic acid",
    "Ash",
    "Alcalinity of ash",
    "Magnesium",
    "Total phenols",
    "Flavanoids",
    "Nonflavanoid phenols",
    "Proanthocyanins",
    "Color intensity",
    "Hue",
    "OD280/OD315 of diluted wines",
    "Proline",
]
FEATURE_COLUMNS = WINE_COLUMNS[1:]

# Centralize search ranges and output names to keep the script/report
aligned.
KMEANS_K_VALUES = list(range(2, 11))
AGGLOMERATIVE_LINKAGES = ["ward", "complete", "average", "single"]
DBSCAN_EPS_VALUES = np.round(np.arange(0.3, 5.01, 0.1), 2)
DBSCAN_MIN_SAMPLES_VALUES = [3, 4, 5, 6, 8, 10]
DBSCAN_REASONABLE_NOISE_RATIO = 0.40

REQUIRED_RESULT_FILES = [
    "data_basic_info.csv",
    "feature_descriptive_statistics.csv",
    "missing_duplicate_check.csv",
    "kmeans_k_selection_results.csv",
    "agglomerative_grid_results.csv",
    "dbscan_grid_search_results.csv",
    "final_model_comparison.csv",
    "model_ranking_summary.csv",
    "cluster_profile_summary.csv",
    "cluster_profile_scaled_mean.csv",
```

```
"cluster_profile_interpretation.txt",
"final_cluster_assignments.csv",
"optional_external_label_comparison.csv",
"optional_kmeans_class_crosstab.csv",
"optional_agglomerative_class_crosstab.csv",
"optional_dbscan_class_crosstab.csv",
]
```

```
REQUIRED_FIGURE_FILES = [
    "01_feature_boxplots_before_scaling.png",
    "02_feature_boxplots_after_scaling.png",
    "03_feature_correlation_heatmap.png",
    "04_pca_original_labels_optional.png",
    "05_kmeans_elbow_curve.png",
    "06_kmeans_silhouette_by_k.png",
    "07_kmeans_ch_by_k.png",
    "08_kmeans_db_by_k.png",
    "09_pca_kmeans_k3_clusters.png",
    "10_pca_best_kmeans_clusters.png",
    "11_agglomerative_silhouette_comparison.png",
    "12_agglomerative_ch_comparison.png",
    "13_agglomerative_db_comparison.png",
    "14_agglomerative_dendrogram.png",
    "15_pca_agglomerative_ward_k3_clusters.png",
    "16_dbscan_k_distance_curve.png",
    "17_dbscan_silhouette_heatmap.png",
    "18_dbscan_noise_ratio_heatmap.png",
    "19_dbscan_cluster_count_heatmap.png",
    "20_pca_best_dbscan_clusters.png",
    "21_final_model_metrics_comparison.png",
    "22_cluster_size_comparison.png",
    "23_cluster_profile_heatmap.png",
    "24_optional_external_metric_comparison.png",
]
```

```
def resolve_paths(exp4_dir: str | Path | None = None) -> dict[str, Path]:
    """Resolve paths so the script works from the repo root or Experiment
    4."""
    if exp4_dir is None:
        exp4_dir = Path(__file__).resolve().parent
```

```

else:
    exp4_dir = Path(exp4_dir).resolve()
    # Allow callers to pass either the repository root or the Experiment
4 folder.
    if exp4_dir.name != "实验四" and (exp4_dir / "实验四").exists():
        exp4_dir = exp4_dir / "实验四"

project_root = exp4_dir.parent
data_dir = exp4_dir / "data"
return {
    "project_root": project_root,
    "exp4_dir": exp4_dir,
    "data_dir": data_dir,
    "results_dir": exp4_dir / "results",
    "figures_dir": exp4_dir / "figures",
    "wine_data": data_dir / "wine.data",
    "wine_names": data_dir / "wine.names",
    "wine_index": data_dir / "Index",
}

def ensure_output_dirs(paths: dict[str, Path]) -> None:
    paths["data_dir"].mkdir(parents=True, exist_ok=True)
    paths["results_dir"].mkdir(parents=True, exist_ok=True)
    paths["figures_dir"].mkdir(parents=True, exist_ok=True)

def extract_wine_zip_if_needed(paths: dict[str, Path]) -> Path | None:
    """Extract wine.data, wine.names, and Index from wine.zip when
needed."""
    if paths["wine_data"].exists():
        return None

    # The assignment forbids downloading data, so only local wine.zip files
are used.
    zip_candidates = [
        paths["data_dir"] / "wine.zip",
        paths["exp4_dir"] / "wine.zip",
        paths["project_root"] / "wine.zip",
    ]
    allowed_names = {"wine.data", "wine.names", "Index"}

```

```

for zip_path in zip_candidates:
    if not zip_path.exists():
        continue

    with zipfile.ZipFile(zip_path) as archive:
        for member in archive.infolist():
            member_name = Path(member.filename).name
            if member.is_dir() or member_name not in allowed_names:
                continue
            target_path = paths["data_dir"] / member_name
            with archive.open(member) as source,
target_path.open("wb") as target:
                shutil.copyfileobj(source, target)

            if paths["wine_data"].exists():
                return zip_path

return None

def check_wine_data(paths: dict[str, Path]) -> None:
    extract_wine_zip_if_needed(paths)
    if not paths["wine_data"].exists():
        raise FileNotFoundError(
            "wine.data was not found. Please put the file at "
            f"{paths['wine_data']}.\\n"
            "If you have wine.zip, place it in the repository root, "
            "Experiment 4 directory, or Experiment 4/data, then run again."
        )

def load_wine_data(paths: dict[str, Path]) -> tuple[pd.DataFrame,
pd.DataFrame, pd.Series]:
    """Load wine.data and remove the class label from the clustering
    features."""
    check_wine_data(paths)

    df = pd.read_csv(paths["wine_data"], header=None)
    if df.shape[1] != len(WINE_COLUMNS):
        raise ValueError(

```

```

        f"Expected {len(WINE_COLUMNS)} columns in wine.data, got
{df.shape[1]})."
    )

    df.columns = WINE_COLUMNS
    df = df.apply(pd.to_numeric, errors="coerce")

    # The original class label is saved only for optional post-clustering
comparison.
    y_true_optional = df["class"].copy()
    # All clustering models must use only the 13 chemical features.
    X = df.drop(columns=["class"]).copy()

    if "class" in X.columns:
        raise RuntimeError("The class label was not removed from X.")

    return df, X, y_true_optional

def standardize_and_project(
    X: pd.DataFrame,
) -> tuple[StandardScaler, pd.DataFrame, pd.DataFrame, PCA]:
    """Standardize feature matrix and build a two-dimensional PCA view."""
    scaler = StandardScaler()
    X_scaled_array = scaler.fit_transform(X)
    X_scaled = pd.DataFrame(X_scaled_array, columns=X.columns,
index=X.index)

    # PCA is a visualization aid only; it is not used for model selection.
    pca = PCA(n_components=2, random_state=RANDOM_STATE)
    X_pca_2d = pca.fit_transform(X_scaled)
    X_pca_2d = pd.DataFrame(X_pca_2d, columns=["PC1", "PC2"],
index=X.index)

    return scaler, X_scaled, X_pca_2d, pca

def save_data_check_outputs(
    df: pd.DataFrame,
    X: pd.DataFrame,
    y_true_optional: pd.Series,

```

```

X_scaled: pd.DataFrame,
paths: dict[str, Path],
) -> list[Path]:
    """Save shape, missing value, duplicate, dtype, and feature
statistics."""
    # Class distribution is recorded as optional context, not as a training
signal.
    class_distribution = y_true_optional.value_counts().sort_index()

    info_rows = [
        {"item": "dataset_file", "value": str(paths["wine_data"])},
        {"item": "n_samples", "value": len(df)},
        {"item": "n_original_columns", "value": df.shape[1]},
        {"item": "n_features_used_for_clustering", "value": X.shape[1]},
        {"item": "removed_label_column", "value": "class"},
        {
            "item": "feature_columns_used_for_clustering",
            "value": "; ".join(X.columns),
        },
        {
            "item": "optional_original_class_distribution_note",
            "value": (
                "Original class labels are optional information only and
are not "
                "used for clustering training, parameter selection, or main
evaluation."
            ),
        },
    ]

    for label, count in class_distribution.items():
        info_rows.append(
            {
                "item": f"optional_original_class_{int(label)}_count",
                "value": int(count),
            }
        )

    for column, dtype in df.dtypes.items():
        info_rows.append({"item": f"dtype_{column}", "value":
str(dtype)})

```

```

data_basic_info = pd.DataFrame(info_rows)
data_basic_info_path = paths["results_dir"] / "data_basic_info.csv"
data_basic_info.to_csv(data_basic_info_path, index=False)

missing_rows = [
    {"item": "total_missing_values_in_original_data", "value":
int(df.isna().sum().sum())},
    {"item": "total_missing_values_in_X", "value":
int(X.isna().sum().sum())},
    {"item": "duplicate_rows_in_original_data", "value":
int(df.duplicated().sum())},
    {"item": "duplicate_rows_in_X", "value":
int(X.duplicated().sum())},
]
for column in df.columns:
    missing_rows.append(
        {"item": f"missing_values_{column}", "value":
int(df[column].isna().sum())}
    )
missing_duplicate_check = pd.DataFrame(missing_rows)
missing_duplicate_path = paths["results_dir"] /
"missing_duplicate_check.csv"
missing_duplicate_check.to_csv(missing_duplicate_path, index=False)

before_stats = X.describe().T.reset_index().rename(columns={"index":
"feature"})
before_stats.insert(0, "data_version", "before_scaling")
after_stats =
X_scaled.describe().T.reset_index().rename(columns={"index":
"feature"})
after_stats.insert(0, "data_version", "after_scaling")
descriptive_statistics = pd.concat([before_stats, after_stats],
ignore_index=True)
descriptive_statistics_path = paths["results_dir"] /
"feature_descriptive_statistics.csv"
descriptive_statistics.to_csv(descriptive_statistics_path,
index=False)

return [
    data_basic_info_path,

```

```

        descriptive_statistics_path,
        missing_duplicate_path,
    ]

def save_figure(fig: plt.Figure, output_path: Path) -> Path:
    fig.tight_layout()
    fig.savefig(output_path, dpi=FIGURE_DPI, bbox_inches="tight")
    plt.close(fig)
    return output_path

def plot_feature_boxplots(
    frame: pd.DataFrame,
    title: str,
    ylabel: str,
    output_path: Path,
) -> Path:
    fig, ax = plt.subplots(figsize=(11, 6))
    boxplot_kwargs = {"patch_artist": True, "showfliers": False}
    try:
        # Matplotlib 3.9 renamed labels to tick_labels; keep a fallback for
        older versions.
        box = ax.boxplot(
            frame.to_numpy(),
            tick_labels=frame.columns,
            **boxplot_kwargs,
        )
    except TypeError:
        box = ax.boxplot(
            frame.to_numpy(),
            labels=frame.columns,
            **boxplot_kwargs,
        )
    for patch in box["boxes"]:
        patch.set_facecolor("#7AA6C2")
        patch.set_alpha(0.75)
    ax.set_title(title)
    ax.set_ylabel(ylabel)
    ax.tick_params(axis="x", labelrotation=70, labelsiz=8)
    ax.grid(axis="y", alpha=0.25)

```



```

    return save_figure(fig, output_path)

def plot_feature_correlation_heatmap(X: pd.DataFrame, paths: dict[str,
Path]) -> Path:
    correlation = X.corr(numeric_only=True)

    fig, ax = plt.subplots(figsize=(9, 7.5))
    image = ax.imshow(correlation, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_title("Feature Correlation Heatmap")
    ax.set_xticks(np.arange(len(correlation.columns)))
    ax.set_yticks(np.arange(len(correlation.index)))
    ax.set_xticklabels(correlation.columns, rotation=70, ha="right",
fontsize=7)
    ax.set_yticklabels(correlation.index, fontsize=7)
    cbar = fig.colorbar(image, ax=ax, shrink=0.85)
    cbar.set_label("Correlation")

    return save_figure(
        fig,
        paths["figures_dir"] / "03_feature_correlation_heatmap.png",
    )

def plot_pca_labels(
    X_pca_2d: pd.DataFrame,
    labels: np.ndarray | pd.Series,
    title: str,
    output_path: Path,
    xlabel: str = "Principal Component 1",
    ylabel: str = "Principal Component 2",
) -> Path:
    fig, ax = plt.subplots(figsize=(7.5, 5.8))
    label_series = pd.Series(labels, index=X_pca_2d.index, name="label")
    unique_labels = sorted(label_series.dropna().unique())
    cmap = plt.get_cmap("tab10")

    for index, label in enumerate(unique_labels):
        mask = label_series == label
        # Noise is shown only for DBSCAN; all other labels are cluster IDs.

```

```

        display_label = "Noise" if int(label) == -1 else f"Cluster
{int(label)}"
        if "original" in output_path.name:
            display_label = f"Class {int(label)}"
        color = "#808080" if int(label) == -1 else cmap(index % 10)
        ax.scatter(
            X_pca_2d.loc[mask, "PC1"],
            X_pca_2d.loc[mask, "PC2"],
            s=38,
            alpha=0.82,
            label=display_label,
            color=color,
            edgecolors="white",
            linewidths=0.35,
        )

    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    ax.grid(alpha=0.2)
    ax.legend(loc="best", fontsize=8, frameon=True)
    return save_figure(fig, output_path)

def plot_data_overview_figures(
    X: pd.DataFrame,
    X_scaled: pd.DataFrame,
    X_pca_2d: pd.DataFrame,
    y_true_optional: pd.Series,
    paths: dict[str, Path],
) -> list[Path]:
    figure_paths = [
        plot_feature_boxplots(
            X,
            "Feature Boxplots Before Scaling",
            "Original Feature Value",
            paths["figures_dir"] /
"01_feature_boxplots_before_scaling.png",
        ),
        plot_feature_boxplots(
            X_scaled,

```

```

        "Feature Boxplots After Standard Scaling",
        "Standardized Feature Value",
        paths["figures_dir"] /
"02_feature_boxplots_after_scaling.png",
    ),
    plot_feature_correlation_heatmap(X, paths),
    plot_pca_labels(
        X_pca_2d,
        y_true_optional,
        "PCA Projection Colored by Original Labels (Optional)",
        paths["figures_dir"] /
"04_pca_original_labels_optional.png",
    ),
]
return figure_paths


def prepare_data(paths: dict[str, Path]) -> dict[str, Any]:
    """Read data, remove class labels, standardize features, and save
checks."""
    ensure_output_dirs(paths)
    df, X, y_true_optional = load_wine_data(paths)
    # From this point on, model inputs are based on X only, never on
df["class"].
    scaler, X_scaled, X_pca_2d, pca = standardize_and_project(X)

    result_paths = save_data_check_outputs(df, X, y_true_optional,
X_scaled, paths)
    figure_paths = plot_data_overview_figures(
        X,
        X_scaled,
        X_pca_2d,
        y_true_optional,
        paths,
    )

    return {
        "data": {
            "df": df,
            "X": X,
            "y_true_optional": y_true_optional,

```

```

        "scaler": scaler,
        "X_scaled": X_scaled,
        "X_pca_2d": X_pca_2d,
        "pca": pca,
    },
    "result_paths": result_paths,
    "figure_paths": figure_paths,
}

def format_cluster_size_summary(labels: np.ndarray) -> str:
    counts = pd.Series(labels).value_counts().sort_index()
    return "; ".join(f"{int(cluster)}:{int(count)}" for cluster, count in
counts.items())

def count_clusters_excluding_noise(labels: np.ndarray) -> int:
    return len([label for label in np.unique(labels) if int(label) != -1])

def compute_internal_metrics(
    X_scaled_array: np.ndarray,
    labels: np.ndarray,
    exclude_noise: bool = False,
) -> dict[str, float]:
    """Compute internal clustering metrics; optionally ignore DBSCAN
noise."""
    labels = np.asarray(labels)
    if exclude_noise:
        # DBSCAN metrics are computed on non-noise samples when valid
clusters exist.
        mask = labels != -1
        X_eval = X_scaled_array[mask]
        labels_eval = labels[mask]
    else:
        X_eval = X_scaled_array
        labels_eval = labels

    unique_labels = np.unique(labels_eval)
    if len(unique_labels) < 2 or len(unique_labels) >= len(labels_eval):
        # Internal metrics are undefined for fewer than two usable clusters.

```

```

        return {
            "silhouette_score": np.nan,
            "calinski_harabasz_score": np.nan,
            "davies_bouldin_score": np.nan,
        }

    return {
        "silhouette_score": float(silhouette_score(X_eval,
labels_eval)),
        "calinski_harabasz_score": float(calinski_harabasz_score(X_eval,
labels_eval)),
        "davies_bouldin_score": float(davies_bouldin_score(X_eval,
labels_eval)),
    }

def select_best_by_internal_metrics(
    results_df: pd.DataFrame,
    extra_sort_columns: list[str] | None = None,
) -> pd.Series:
    # Tie-breaking follows the report rule: Silhouette high, CH high, DB
low.
    sort_columns = [
        "silhouette_score",
        "calinski_harabasz_score",
        "davies_bouldin_score",
    ]
    ascending = [False, False, True]
    if extra_sort_columns:
        sort_columns.extend(extra_sort_columns)
        ascending.extend([True] * len(extra_sort_columns))

    valid_df = results_df.dropna(subset=["silhouette_score"]).copy()
    if valid_df.empty:
        raise ValueError("No valid clustering result is available for model
selection.")

    return valid_df.sort_values(sort_columns,
ascending=ascending).iloc[0]

```

```

def plot_single_metric_by_k(
    results_df: pd.DataFrame,
    metric: str,
    title: str,
    ylabel: str,
    output_path: Path,
) -> Path:
    fig, ax = plt.subplots(figsize=(7.5, 5))
    ax.plot(
        results_df["n_clusters"],
        results_df[metric],
        marker="o",
        linewidth=2,
        color="#4C78A8",
    )
    ax.set_title(title)
    ax.set_xlabel("Number of Clusters (k)")
    ax.set_ylabel(ylabel)
    ax.set_xticks(results_df["n_clusters"])
    ax.grid(alpha=0.25)
    return save_figure(fig, output_path)

def run_kmeans_experiment(data: dict[str, Any], paths: dict[str, Path])
-> dict[str, Any]:
    X_scaled_array = data["X_scaled"].to_numpy()
    labels_by_k: dict[int, np.ndarray] = {}
    records = []

    # KMeans is evaluated across k values; inertia is kept only for the
    elbow curve.
    for k in KMEANS_K_VALUES:
        model = KMeans(n_clusters=k, n_init=50,
random_state=RANDOM_STATE)
        start_time = time.perf_counter()
        labels = model.fit_predict(X_scaled_array)
        elapsed = time.perf_counter() - start_time

        labels_by_k[k] = labels
        metrics = compute_internal_metrics(X_scaled_array, labels)
        records.append(

```

```

        {
            "model": "KMeans",
            "n_clusters": k,
            "inertia": float(model.inertia_),
            **metrics,
            "training_time_seconds": elapsed,
            "cluster_size_summary":
format_cluster_size_summary(labels),
        }
    )

    results_df = pd.DataFrame(records)
    result_path = paths["results_dir"] / "kmeans_k_selection_results.csv"
    results_df.to_csv(result_path, index=False)

    best_row = select_best_by_internal_metrics(results_df)
    best_k = int(best_row["n_clusters"])

    # Both the assignment-required k=3 result and the best-by-silhouette
result are kept.
    figure_paths = [
        plot_single_metric_by_k(
            results_df,
            "inertia",
            "KMeans Elbow Curve",
            "Inertia",
            paths["figures_dir"] / "05_kmeans_elbow_curve.png",
        ),
        plot_single_metric_by_k(
            results_df,
            "silhouette_score",
            "KMeans Silhouette Score by k",
            "Silhouette Coefficient",
            paths["figures_dir"] / "06_kmeans_silhouette_by_k.png",
        ),
        plot_single_metric_by_k(
            results_df,
            "calinski_harabasz_score",
            "KMeans Calinski-Harabasz Index by k",
            "Calinski-Harabasz Index",

```

```

        paths["figures_dir"] / "07_kmeans_ch_by_k.png",
    ),
    plot_single_metric_by_k(
        results_df,
        "davies_bouldin_score",
        "KMeans Davies-Bouldin Index by k",
        "Davies-Bouldin Index",
        paths["figures_dir"] / "08_kmeans_db_by_k.png",
    ),
    plot_pca_labels(
        data["X_pca_2d"],
        labels_by_k[3],
        "PCA Projection of KMeans Clusters (k=3)",
        paths["figures_dir"] / "09_pca_kmeans_k3_clusters.png",
    ),
    plot_pca_labels(
        data["X_pca_2d"],
        labels_by_k[best_k],
        f"PCA Projection of Best KMeans Clusters (k={best_k})",
        paths["figures_dir"] / "10_pca_best_kmeans_clusters.png",
    ),
]

return {
    "results": results_df,
    "labels_by_k": labels_by_k,
    "best_k": best_k,
    "best_row": best_row,
    "k3_labels": labels_by_k[3],
    "best_labels": labels_by_k[best_k],
    "result_paths": [result_path],
    "figure_paths": figure_paths,
}

```

```

def plot_agglomerative_metric_comparison(
    results_df: pd.DataFrame,
    metric: str,
    title: str,
    ylabel: str,
    output_path: Path,

```



```

) -> Path:
    fig, ax = plt.subplots(figsize=(8, 5.5))
    for linkage in AGGLOMERATIVE_LINKAGES:
        subset = results_df[results_df["linkage"] == linkage]
        ax.plot(
            subset["n_clusters"],
            subset[metric],
            marker="o",
            linewidth=2,
            label=linkage,
        )
    ax.set_title(title)
    ax.set_xlabel("Number of Clusters (k)")
    ax.set_ylabel(ylabel)
    ax.set_xticks(KMEANS_K_VALUES)
    ax.grid(alpha=0.25)
    ax.legend(title="Linkage")
    return save_figure(fig, output_path)

def plot_agglomerative_dendrogram(data: dict[str, Any], paths: dict[str,
Path]) -> Path:
    from scipy.cluster.hierarchy import dendrogram, linkage

    linked = linkage(data["X_scaled"].to_numpy(), method="ward")

    fig, ax = plt.subplots(figsize=(10, 5.5))
    dendrogram(
        linked,
        truncate_mode="level",
        p=5,
        no_labels=True,
        color_threshold=None,
        ax=ax,
    )
    ax.set_title("Agglomerative Clustering Dendrogram (Ward Linkage)")
    ax.set_xlabel("Samples")
    ax.set_ylabel("Distance")
    ax.grid(axis="y", alpha=0.2)
    return save_figure(fig, paths["figures_dir"] /
"14_agglomerative_dendrogram.png")

```

```

def run_agglomerative_experiment(
    data: dict[str, Any],
    paths: dict[str, Path],
) -> dict[str, Any]:
    X_scaled_array = data["X_scaled"].to_numpy()
    labels_by_setting: dict[tuple[str, int], np.ndarray] = {}
    records = []

    # Linkage methods are compared on the same scaled feature matrix for
    # fairness.
    for linkage in AGGLOMERATIVE_LINKAGES:
        for k in KMEANS_K_VALUES:
            model = AgglomerativeClustering(n_clusters=k,
linkage=linkage)
            start_time = time.perf_counter()
            labels = model.fit_predict(X_scaled_array)
            elapsed = time.perf_counter() - start_time

            labels_by_setting[(linkage, k)] = labels
            metrics = compute_internal_metrics(X_scaled_array, labels)
            records.append(
                {
                    "model": "AgglomerativeClustering",
                    "linkage": linkage,
                    "n_clusters": k,
                    **metrics,
                    "training_time_seconds": elapsed,
                    "cluster_size_summary":
format_cluster_size_summary(labels),
                }
            )

    results_df = pd.DataFrame(records)
    result_path = paths["results_dir"] / "agglomerative_grid_results.csv"
    results_df.to_csv(result_path, index=False)

    best_row = select_best_by_internal_metrics(results_df)
    best_linkage = str(best_row["linkage"])
    best_k = int(best_row["n_clusters"])

```

```

# Ward k=3 is retained as the required final hierarchical baseline.
figure_paths = [
    plot_agglomerative_metric_comparison(
        results_df,
        "silhouette_score",
        "Agglomerative Silhouette Score Comparison",
        "Silhouette Coefficient",
        paths["figures_dir"] /
"11_agglomerative_silhouette_comparison.png",
    ),
    plot_agglomerative_metric_comparison(
        results_df,
        "calinski_harabasz_score",
        "Agglomerative Calinski-Harabasz Index Comparison",
        "Calinski-Harabasz Index",
        paths["figures_dir"] / "12_agglomerative_ch_comparison.png",
    ),
    plot_agglomerative_metric_comparison(
        results_df,
        "davies_bouldin_score",
        "Agglomerative Davies-Bouldin Index Comparison",
        "Davies-Bouldin Index",
        paths["figures_dir"] / "13_agglomerative_db_comparison.png",
    ),
    plot_agglomerative_dendrogram(data, paths),
    plot_pca_labels(
        data["X_pca_2d"],
        labels_by_setting[("ward", 3)],
        "PCA Projection of Agglomerative Clusters (Ward, k=3)",
        paths["figures_dir"] /
"15_pca_agglomerative_ward_k3_clusters.png",
    ),
]

return {
    "results": results_df,
    "labels_by_setting": labels_by_setting,
    "best_linkage": best_linkage,
    "best_k": best_k,
}

```

```

        "best_row": best_row,
        "ward_k3_labels": labels_by_setting[("ward", 3)],
        "best_labels": labels_by_setting[(best_linkage, best_k)],
        "result_paths": [result_path],
        "figure_paths": figure_paths,
    }

def plot_dbscan_k_distance_curve(data: dict[str, Any], paths: dict[str,
Path]) -> Path:
    neighbors = NearestNeighbors(n_neighbors=5)
    neighbors.fit(data["X_scaled"].to_numpy())
    distances, _ = neighbors.kneighbors(data["X_scaled"].to_numpy())
    # Sorted kth-neighbor distances help explain a reasonable eps range.
    kth_distances = np.sort(distances[:, -1])

    fig, ax = plt.subplots(figsize=(7.5, 5))
    ax.plot(np.arange(1, len(kth_distances) + 1), kth_distances,
color="#4C78A8")
    ax.set_title("DBSCAN k-Distance Curve (5th Nearest Neighbor)")
    ax.set_xlabel("Samples Sorted by Distance")
    ax.set_ylabel("5th Nearest Neighbor Distance")
    ax.grid(alpha=0.25)
    return save_figure(fig, paths["figures_dir"] /
"16_dbscan_k_distance_curve.png")

def plot_dbscan_heatmap(
    results_df: pd.DataFrame,
    value_column: str,
    title: str,
    colorbar_label: str,
    output_path: Path,
    cmap_name: str = "viridis",
) -> Path:
    # Pivot the grid search result into eps x min_samples matrices for
reporting.
    pivot = results_df.pivot(
        index="min_samples",
        columns="eps",
        values=value_column,

```

```

).sort_index()
matrix = pivot.to_numpy(dtype=float)

fig, ax = plt.subplots(figsize=(10, 4.8))
cmap = plt.get_cmap(cmap_name).copy()
cmap.set_bad("#EFEFEF")
image = ax.imshow(matrix, aspect="auto", cmap=cmap, origin="lower")

x_positions = np.arange(len(pivot.columns))
shown_x_positions = x_positions[::5]
if x_positions[-1] not in shown_x_positions:
    shown_x_positions = np.append(shown_x_positions,
x_positions[-1])
ax.set_xticks(shown_x_positions)
ax.set_xticklabels([f"{pivot.columns[i]:.1f}" for i in
shown_x_positions])
ax.set_yticks(np.arange(len(pivot.index)))
ax.set_yticklabels([str(value) for value in pivot.index])
ax.set_title(title)
ax.set_xlabel("eps")
ax.set_ylabel("min_samples")
cbar = fig.colorbar(image, ax=ax, shrink=0.9)
cbar.set_label(colorbar_label)
return save_figure(fig, output_path)

def select_best_dbscan(results_df: pd.DataFrame) -> pd.Series:
    valid_df = results_df.dropna(subset=["silhouette_score"]).copy()
    # Avoid recommending a DBSCAN setting that labels too many samples as
noise.
    reasonable_df = valid_df[
        valid_df["noise_ratio"] <= DBSCAN_REASONABLE_NOISE_RATIO
    ].copy()
    selection_df = reasonable_df if not reasonable_df.empty else valid_df

    if selection_df.empty:
        return results_df.sort_values(
            ["cluster_count_excluding_noise", "noise_ratio"],
            ascending=[False, True],
        ).iloc[0]

```

```

return selection_df.sort_values(
    [
        "silhouette_score",
        "calinski_harabasz_score",
        "davies_bouldin_score",
        "noise_ratio",
    ],
    ascending=[False, False, True, True],
).iloc[0]

def run_dbSCAN_experiment(data: dict[str, Any], paths: dict[str, Path])
-> dict[str, Any]:
    X_scaled_array = data["X_scaled"].to_numpy()
    labels_by_setting: dict[tuple[float, int], np.ndarray] = {}
    records = []

    # DBSCAN is sensitive to eps/min_samples, so the full grid is retained.
    for eps in DBSCAN_EPS_VALUES:
        for min_samples in DBSCAN_MIN_SAMPLES_VALUES:
            model = DBSCAN(eps=float(eps), min_samples=int(min_samples))
            start_time = time.perf_counter()
            labels = model.fit_predict(X_scaled_array)
            elapsed = time.perf_counter() - start_time

            cluster_count = count_clusters_excluding_noise(labels)
            noise_count = int(np.sum(labels == -1))
            noise_ratio = noise_count / len(labels)

            if cluster_count < 2:
                # Metrics are intentionally NaN when there are not enough
valid clusters.
                metrics = {
                    "silhouette_score": np.nan,
                    "calinski_harabasz_score": np.nan,
                    "davies_bouldin_score": np.nan,
                }
            else:
                metrics = compute_internal_metrics(
                    X_scaled_array,
                    labels,

```

```

        exclude_noise=True,
    )

    labels_by_setting[(float(eps), int(min_samples))] = labels
    records.append(
        {
            "model": "DBSCAN",
            "eps": float(eps),
            "min_samples": int(min_samples),
            "cluster_count_excluding_noise": cluster_count,
            "noise_count": noise_count,
            "noise_ratio": noise_ratio,
            **metrics,
            "training_time_seconds": elapsed,
        }
    )

results_df = pd.DataFrame(records)
result_path = paths["results_dir"] / "dbscan_grid_search_results.csv"
results_df.to_csv(result_path, index=False)

best_row = select_best_dbscan(results_df)
best_eps = float(best_row["eps"])
best_min_samples = int(best_row["min_samples"])

figure_paths = [
    plot_dbscan_k_distance_curve(data, paths),
    plot_dbscan_heatmap(
        results_df,
        "silhouette_score",
        "DBSCAN Silhouette Score Heatmap",
        "Silhouette Coefficient",
        paths["figures_dir"] / "17_dbscan_silhouette_heatmap.png",
    ),
    plot_dbscan_heatmap(
        results_df,
        "noise_ratio",
        "DBSCAN Noise Ratio Heatmap",
        "Noise Ratio",
        paths["figures_dir"] / "18_dbscan_noise_ratio_heatmap.png",
    ),

```

```

        cmap_name="magma",
    ),
    plot_dbscan_heatmap(
        results_df,
        "cluster_count_excluding_noise",
        "DBSCAN Cluster Count Heatmap",
        "Cluster Count Excluding Noise",
        paths["figures_dir"] /
"19_dbscan_cluster_count_heatmap.png",
        cmap_name="plasma",
    ),
    plot_pca_labels(
        data["X_pca_2d"],
        labels_by_setting[(best_eps, best_min_samples)],
        f"PCA Projection of Best DBSCAN Clusters (eps={best_eps:.2f},
min_samples={best_min_samples})",
        paths["figures_dir"] / "20_pca_best_dbscan_clusters.png",
    ),
]

return {
    "results": results_df,
    "labels_by_setting": labels_by_setting,
    "best_eps": best_eps,
    "best_min_samples": best_min_samples,
    "best_row": best_row,
    "best_labels": labels_by_setting[(best_eps, best_min_samples)],
    "result_paths": [result_path],
    "figure_paths": figure_paths,
}

def make_final_model_comparison(
    kmeans_outputs: dict[str, Any],
    agglomerative_outputs: dict[str, Any],
    dbscan_outputs: dict[str, Any],
) -> pd.DataFrame:
    # The final table contains required fixed settings plus internally
selected models.
    kmeans_results = kmeans_outputs["results"]
    agglomerative_results = agglomerative_outputs["results"]

```



```

dbscan_best = dbscan_outputs["best_row"]

kmeans_k3 = kmeans_results[kmeans_results["n_clusters"] == 3].iloc[0]
best_kmeans = kmeans_outputs["best_row"]
agglomerative_ward_k3 = agglomerative_results[
    (agglomerative_results["linkage"] == "ward")
    & (agglomerative_results["n_clusters"] == 3)
].iloc[0]
best_agglomerative = agglomerative_outputs["best_row"]

rows = [
    {
        "model": "KMeans",
        "setting": "k=3",
        "n_clusters": int(kmeans_k3["n_clusters"]),
        "noise_ratio": 0.0,
        "silhouette_score": kmeans_k3["silhouette_score"],
        "calinski_harabasz_score":
kmeans_k3["calinski_harabasz_score"],
        "davies_bouldin_score": kmeans_k3["davies_bouldin_score"],
        "training_time_seconds":
kmeans_k3["training_time_seconds"],
        "notes": "Fixed k=3 retained for cluster interpretation.",
    },
    {
        "model": "KMeans",
        "setting": f"best by silhouette,
k={int(best_kmeans['n_clusters'])}",
        "n_clusters": int(best_kmeans["n_clusters"]),
        "noise_ratio": 0.0,
        "silhouette_score": best_kmeans["silhouette_score"],
        "calinski_harabasz_score":
best_kmeans["calinski_harabasz_score"],
        "davies_bouldin_score":
best_kmeans["davies_bouldin_score"],
        "training_time_seconds":
best_kmeans["training_time_seconds"],
        "notes": "Selected by internal clustering metrics only.",
    },
    {

```

```

        "model": "AgglomerativeClustering",
        "setting": "linkage=ward, k=3",
        "n_clusters": int(agglomerative_ward_k3["n_clusters"]),
        "noise_ratio": 0.0,
        "silhouette_score":
agglomerative_ward_k3["silhouette_score"],
        "calinski_harabasz_score": agglomerative_ward_k3[
            "calinski_harabasz_score"
        ],
        "davies_bouldin_score":
agglomerative_ward_k3["davies_bouldin_score"],
        "training_time_seconds":
agglomerative_ward_k3["training_time_seconds"],
        "notes": "Ward linkage with k=3 retained as a hierarchical
baseline.",
    },
    {
        "model": "AgglomerativeClustering",
        "setting": (
            "best by silhouette, "
            f"linkage={best_agglomerative['linkage']}, "
            f"k={int(best_agglomerative['n_clusters'])}"
        ),
        "n_clusters": int(best_agglomerative["n_clusters"]),
        "noise_ratio": 0.0,
        "silhouette_score": best_agglomerative["silhouette_score"],
        "calinski_harabasz_score": best_agglomerative[
            "calinski_harabasz_score"
        ],
        "davies_bouldin_score":
best_agglomerative["davies_bouldin_score"],
        "training_time_seconds":
best_agglomerative["training_time_seconds"],
        "notes": "Selected by internal clustering metrics only.",
    },
    {
        "model": "DBSCAN",
        "setting": (
            f"eps={float(dbscan_best['eps']):.2f}, "
            f"min_samples={int(dbscan_best['min_samples'])}"

```

```

    ),
    "n_clusters":
int(dbscan_best["cluster_count_excluding_noise"]),
    "noise_ratio": float(dbscan_best["noise_ratio"]),
    "silhouette_score": dbscan_best["silhouette_score"],
    "calinski_harabasz_score":
dbscan_best["calinski_harabasz_score"],
    "davies_bouldin_score":
dbscan_best["davies_bouldin_score"],
    "training_time_seconds":
dbscan_best["training_time_seconds"],
    "notes": (
        "Best valid DBSCAN setting; noise ratio is considered
because "
        "DBSCAN is sensitive to eps and min_samples."
    ),
    },
]
final_df = pd.DataFrame(rows)
return sort_final_models(final_df)

def sort_final_models(final_df: pd.DataFrame) -> pd.DataFrame:
    working_df = final_df.copy()
    # High-noise DBSCAN settings are not preferred even when one metric
looks strong.
    dbscan_high_noise = (
        (working_df["model"] == "DBSCAN")
        & (working_df["noise_ratio"] > DBSCAN_REASONABLE_NOISE_RATIO)
    )
    working_df["_recommendation_candidate"] = ~dbscan_high_noise
    working_df["_silhouette_sort"] =
working_df["silhouette_score"].fillna(-np.inf)
    working_df["_ch_sort"] =
working_df["calinski_harabasz_score"].fillna(-np.inf)
    working_df["_db_sort"] =
working_df["davies_bouldin_score"].fillna(np.inf)

    sorted_df = working_df.sort_values(
        [
            "_recommendation_candidate",

```

```

        "_silhouette_sort",
        "_ch_sort",
        "_db_sort",
        "noise_ratio",
    ],
    ascending=[False, False, False, True, True],
)
return sorted_df[final_df.columns].reset_index(drop=True)

def make_model_ranking_summary(final_df: pd.DataFrame) -> pd.DataFrame:
    ranking_df = final_df.copy()
    ranking_df.insert(0, "rank", np.arange(1, len(ranking_df) + 1))
    ranking_df["recommendation_candidate"] = ~(
        (ranking_df["model"] == "DBSCAN")
        & (ranking_df["noise_ratio"] > DBSCAN_REASONABLE_NOISE_RATIO)
    )
    ranking_df["ranking_rule"] = (
        "Prefer higher Silhouette, then higher Calinski-Harabasz, then
lower "
        "Davies-Bouldin. DBSCAN settings with high noise ratio are not
preferred."
    )
    ranking_df["recommended"] = ranking_df["rank"] == 1
    return ranking_df

def save_final_cluster_assignments(
    kmeans_outputs: dict[str, Any],
    agglomerative_outputs: dict[str, Any],
    dbscan_outputs: dict[str, Any],
    paths: dict[str, Path],
) -> Path:
    n_samples = len(kmeans_outputs["k3_labels"])
    # Do not include the original class label in this main clustering output.
    assignments = pd.DataFrame(
        {
            "sample_id": np.arange(1, n_samples + 1),
            "kmeans_k3_cluster": kmeans_outputs["k3_labels"],
            "best_kmeans_cluster": kmeans_outputs["best_labels"],

```

```

        "agglomerative_ward_k3_cluster":
agglomerative_outputs["ward_k3_labels"],
        "best_agglomerative_cluster":
agglomerative_outputs["best_labels"],
        "best_dbscan_cluster": dbscan_outputs["best_labels"],
    }
)
output_path = paths["results_dir"] / "final_cluster_assignments.csv"
assignments.to_csv(output_path, index=False)
return output_path

def save_cluster_profiles(
    data: dict[str, Any],
    kmeans_outputs: dict[str, Any],
    paths: dict[str, Path],
) -> dict[str, Any]:
    # KMeans k=3 is used for interpretable cluster profiles in both feature
spaces.
    labels = kmeans_outputs["k3_labels"]
    X = data["X"]
    X_scaled = data["X_scaled"]

    profile_frame = X.copy()
    profile_frame["cluster"] = labels
    sample_counts =
profile_frame.groupby("cluster").size().rename("sample_count")
    original_means = profile_frame.groupby("cluster")[X.columns].mean()
    profile_summary = pd.concat([sample_counts, original_means],
axis=1).reset_index()

    profile_summary_path = paths["results_dir"] /
"cluster_profile_summary.csv"
    profile_summary.to_csv(profile_summary_path, index=False)

    scaled_profile_frame = X_scaled.copy()
    scaled_profile_frame["cluster"] = labels
    scaled_means =
scaled_profile_frame.groupby("cluster")[X.columns].mean().reset_index(
)

```

```

scaled_mean_path = paths["results_dir"] /
"cluster_profile_scaled_mean.csv"
scaled_means.to_csv(scaled_mean_path, index=False)

interpretation_text = build_cluster_profile_interpretation(
    profile_summary,
    scaled_means,
)
interpretation_path = paths["results_dir"] /
"cluster_profile_interpretation.txt"
interpretation_path.write_text(interpretation_text,
encoding="utf-8")

heatmap_path = plot_cluster_profile_heatmap(
    scaled_means,
    paths["figures_dir"] / "23_cluster_profile_heatmap.png",
)

return {
    "profile_summary": profile_summary,
    "scaled_means": scaled_means,
    "interpretation_text": interpretation_text,
    "result_paths": [
        profile_summary_path,
        scaled_mean_path,
        interpretation_path,
    ],
    "figure_paths": [heatmap_path],
}

def build_cluster_profile_interpretation(
    profile_summary: pd.DataFrame,
    scaled_means: pd.DataFrame,
) -> str:
    lines = [
        "Cluster profile interpretation based on KMeans k=3.",
        "The values are interpreted from standardized feature means;
positive values are above the overall average and negative values are below
the overall average.",
    ]

```

```

scaled_indexed = scaled_means.set_index("cluster")
profile_indexed = profile_summary.set_index("cluster")

# The top positive and negative standardized means give a compact
description.
for cluster_id, row in scaled_indexed.iterrows():
    feature_values = row.drop(labels=[],
errors="ignore").astype(float)
    high_features =
feature_values.sort_values(ascending=False).head(3)
    low_features =
feature_values.sort_values(ascending=True).head(3)
    sample_count = int(profile_indexed.loc[cluster_id,
"sample_count"])
    high_text = ", ".join(
        f"{feature} ({value:+.2f})" for feature, value in
high_features.items()
    )
    low_text = ", ".join(
        f"{feature} ({value:+.2f})" for feature, value in
low_features.items()
    )
    lines.append(
        f"Cluster {int(cluster_id)} contains {sample_count} samples.
"
        f"It is relatively high in {high_text}, and relatively low in
{low_text}."
    )

    return "\n".join(lines)

def plot_cluster_profile_heatmap(scaled_means: pd.DataFrame, output_path:
Path) -> Path:
    heatmap_data = scaled_means.set_index("cluster")
    values = heatmap_data.to_numpy(dtype=float)
    max_abs = max(1.0, float(np.nanmax(np.abs(values))))

    fig, ax = plt.subplots(figsize=(11, 4.6))
    image = ax.imshow(values, aspect="auto", cmap="coolwarm",
vmin=-max_abs, vmax=max_abs)

```

```

ax.set_title("KMeans k=3 Cluster Profile Heatmap")
ax.set_xlabel("Feature")
ax.set_ylabel("Cluster")
ax.set_xticks(np.arange(len(heatmap_data.columns)))
ax.set_xticklabels(heatmap_data.columns, rotation=70, ha="right",
fontsize=8)
ax.set_yticks(np.arange(len(heatmap_data.index)))
ax.set_yticklabels([f"Cluster {int(value)}" for value in
heatmap_data.index])
cbar = fig.colorbar(image, ax=ax, shrink=0.9)
cbar.set_label("Standardized Mean")
return save_figure(fig, output_path)

def save_optional_external_label_comparison(
    data: dict[str, Any],
    kmeans_outputs: dict[str, Any],
    agglomerative_outputs: dict[str, Any],
    dbscan_outputs: dict[str, Any],
    paths: dict[str, Path],
) -> list[Path]:
    """Use original labels only after clustering as optional external
comparison."""
    # This section is deliberately separate from training, selection, and
main metrics.
    y_true_optional = data["y_true_optional"].to_numpy()
    label_sets = {
        "KMeans k=3": kmeans_outputs["k3_labels"],
        "Best KMeans": kmeans_outputs["best_labels"],
        "Agglomerative ward k=3":
agglomerative_outputs["ward_k3_labels"],
        "Best Agglomerative": agglomerative_outputs["best_labels"],
        "Best DBSCAN": dbscan_outputs["best_labels"],
    }

    rows = []
    for model_name, labels in label_sets.items():
        rows.append(
            {
                "model": model_name,
                "adjusted_rand_index_optional": adjusted_rand_score(

```



```

        y_true_optional,
        labels,
    ),
    "normalized_mutual_info_optional":
normalized_mutual_info_score(
    y_true_optional,
    labels,
),
    "homogeneity_optional":
homogeneity_score(y_true_optional, labels),
    "completeness_optional":
completeness_score(y_true_optional, labels),
    "v_measure_optional": v_measure_score(y_true_optional,
labels),
    "note": (
        "Optional external comparison only; original labels
were not "
        "used in clustering training, parameter selection, or
main evaluation."
    ),
}
)

comparison_df = pd.DataFrame(rows)
comparison_path = paths["results_dir"] /
"optional_external_label_comparison.csv"
comparison_df.to_csv(comparison_path, index=False)

optional_paths = [comparison_path]
crosstab_specs = [
    (
        "optional_kmeans_class_crosstab.csv",
        kmeans_outputs["k3_labels"],
    ),
    (
        "optional_agglomerative_class_crosstab.csv",
        agglomerative_outputs["ward_k3_labels"],
    ),
    (
        "optional_dbscan_class_crosstab.csv",

```

```

        dbscan_outputs["best_labels"],
    ),
]
for file_name, labels in crosstab_specs:
    crosstab = pd.crosstab(
        pd.Series(labels, name="cluster"),
        pd.Series(y_true_optional, name="class"),
    )
    crosstab_path = paths["results_dir"] / file_name
    crosstab.to_csv(crosstab_path)
    optional_paths.append(crosstab_path)

return optional_paths

def plot_optional_external_metric_comparison(
    optional_comparison: pd.DataFrame,
    output_path: Path,
) -> Path:
    metric_columns = [
        "adjusted_rand_index_optional",
        "normalized_mutual_info_optional",
        "homogeneity_optional",
        "completeness_optional",
        "v_measure_optional",
    ]
    display_metrics = ["ARI", "NMI", "Homogeneity", "Completeness",
"V-measure"]
    model_labels = optional_comparison["model"].tolist()
    x_positions = np.arange(len(model_labels))
    bar_width = 0.15

    fig, ax = plt.subplots(figsize=(10, 5.5))
    for metric_index, metric in enumerate(metric_columns):
        offset = (metric_index - 2) * bar_width
        ax.bar(
            x_positions + offset,
            optional_comparison[metric],
            width=bar_width,
            label=display_metrics[metric_index],
        )

```

```

    ax.set_title("Optional External Label Comparison Metrics")
    ax.set_xlabel("Model")
    ax.set_ylabel("Score")
    ax.set_xticks(x_positions)
    ax.set_xticklabels(model_labels, rotation=25, ha="right")
    ax.set_ylim(0, 1.05)
    ax.grid(axis="y", alpha=0.25)
    ax.legend(ncol=3, fontsize=8)
    return save_figure(fig, output_path)

def plot_final_model_metrics_comparison(
    final_df: pd.DataFrame,
    output_path: Path,
) -> Path:
    model_labels = [
        f"{row.model}\n{row.setting}" for row in
final_df.itertuples(index=False)
    ]
    x_positions = np.arange(len(model_labels))

    fig, axes = plt.subplots(3, 1, figsize=(10, 9), sharex=True)
    metric_specs = [
        ("silhouette_score", "Silhouette Coefficient", "Higher is
better"),
        ("calinski_harabasz_score", "Calinski-Harabasz Index", "Higher is
better"),
        ("davies_bouldin_score", "Davies-Bouldin Index", "Lower is
better"),
    ]
    colors = ["#4C78A8", "#59A14F", "#F58518"]

    # Plot the three internal metrics separately because their scales are
different.
    for ax, (metric, ylabel, subtitle), color in zip(axes, metric_specs,
colors):
        ax.bar(x_positions, final_df[metric], color=color, alpha=0.85)
        ax.set_ylabel(ylabel)
        ax.set_title(subtitle, fontsize=10)
        ax.grid(axis="y", alpha=0.25)

```

```

    axes[0].set_title("Final Model Internal Metrics Comparison",
fontsize=13)
    axes[-1].set_xticks(x_positions)
    axes[-1].set_xticklabels(model_labels, rotation=20, ha="right",
fontsize=8)
    return save_figure(fig, output_path)

def plot_cluster_size_comparison(
    labels_map: dict[str, np.ndarray],
    output_path: Path,
) -> Path:
    # Stacked horizontal bars make cluster balance and DBSCAN noise easy
to compare.
    all_cluster_labels = sorted(
        {int(label) for labels in labels_map.values() for label in
np.unique(labels)}
    )
    colors = plt.get_cmap("tab20")(np.linspace(0, 1, max(1,
len(all_cluster_labels))))
    color_map = {
        cluster_label: colors[index]
        for index, cluster_label in enumerate(all_cluster_labels)
    }

    fig, ax = plt.subplots(figsize=(10, 5.4))
    y_positions = np.arange(len(labels_map))
    for y_position, (model_name, labels) in zip(y_positions,
labels_map.items()):
        counts = pd.Series(labels).value_counts().sort_index()
        left = 0
        for cluster_label in all_cluster_labels:
            count = int(counts.get(cluster_label, 0))
            if count == 0:
                continue
            legend_label = "Noise" if cluster_label == -1 else f"Cluster
{cluster_label}"
            ax.barh(
                y_position,
                count,

```

```

        left=left,
        color=color_map[cluster_label],
        label=legend_label,
    )
    left += count

handles, labels = ax.get_legend_handles_labels()
unique_legend = dict(zip(labels, handles))
ax.set_title("Cluster Size Comparison")
ax.set_xlabel("Sample Count")
ax.set_yticks(y_positions)
ax.set_yticklabels(list(labels_map.keys()))
ax.grid(axis="x", alpha=0.25)
ax.legend(
    unique_legend.values(),
    unique_legend.keys(),
    bbox_to_anchor=(1.01, 1),
    loc="upper left",
    fontsize=8,
)
return save_figure(fig, output_path)

def build_short_analysis_text(final_df: pd.DataFrame) -> str:
    recommended = final_df.iloc[0]
    dbscan_row = final_df[final_df["model"] == "DBSCAN"]
    dbscan_note = ""
    if not dbscan_row.empty:
        dbscan_noise_ratio = float(dbscan_row.iloc[0]["noise_ratio"])
        dbscan_note = (
            f" The selected DBSCAN setting has a noise ratio of "
            f"{dbscan_noise_ratio:.2%}, "
            "so DBSCAN sensitivity to eps and min_samples should be "
            "discussed."
        )

    return (
        f"Recommended model: {recommended['model']}"
        f"({recommended['setting']}). "
        f"It has Silhouette={recommended['silhouette_score']:.4f}, "

```

```

        f"Calinski-Harabasz={recommended['calinski_harabasz_score']:.4f}
    , "
        f"and Davies-Bouldin={recommended['davies_bouldin_score']:.4f}.
    "

    "The recommendation is based only on internal clustering metrics;
original "
    "class labels are used only for optional external comparison."
    f"{dbscan_note}"
)

def run_final_analysis(
    data: dict[str, Any],
    kmeans_outputs: dict[str, Any],
    agglomerative_outputs: dict[str, Any],
    dbscan_outputs: dict[str, Any],
    paths: dict[str, Path],
) -> dict[str, Any]:
    # Collect the final tables, cluster assignments, profiles, and optional
label checks.
    final_df = make_final_model_comparison(
        kmeans_outputs,
        agglomerative_outputs,
        dbscan_outputs,
    )
    final_model_path = paths["results_dir"] /
"final_model_comparison.csv"
    final_df.to_csv(final_model_path, index=False)

    ranking_df = make_model_ranking_summary(final_df)
    ranking_path = paths["results_dir"] / "model_ranking_summary.csv"
    ranking_df.to_csv(ranking_path, index=False)

    assignments_path = save_final_cluster_assignments(
        kmeans_outputs,
        agglomerative_outputs,
        dbscan_outputs,
        paths,
    )

    profile_outputs = save_cluster_profiles(data, kmeans_outputs, paths)

```

```

optional_paths = save_optional_external_label_comparison(
    data,
    kmeans_outputs,
    agglomerative_outputs,
    dbscan_outputs,
    paths,
)
optional_comparison = pd.read_csv(
    paths["results_dir"] / "optional_external_label_comparison.csv"
)

labels_map = {
    "KMeans k=3": kmeans_outputs["k3_labels"],
    "Best KMeans": kmeans_outputs["best_labels"],
    "Agglomerative ward k=3":
agglomerative_outputs["ward_k3_labels"],
    "Best Agglomerative": agglomerative_outputs["best_labels"],
    "Best DBSCAN": dbscan_outputs["best_labels"],
}
figure_paths = [
    plot_final_model_metrics_comparison(
        final_df,
        paths["figures_dir"] /
"21_final_model_metrics_comparison.png",
    ),
    plot_cluster_size_comparison(
        labels_map,
        paths["figures_dir"] / "22_cluster_size_comparison.png",
    ),
]
figure_paths.extend(profile_outputs["figure_paths"])
figure_paths.append(
    plot_optional_external_metric_comparison(
        optional_comparison,
        paths["figures_dir"] /
"24_optional_external_metric_comparison.png",
    )
)

result_paths = [

```

```

        final_model_path,
        ranking_path,
        assignments_path,
        *profile_outputs["result_paths"],
        *optional_paths,
    ]
    analysis_text = build_short_analysis_text(final_df)

    return {
        "final_model_comparison": final_df,
        "model_ranking_summary": ranking_df,
        "cluster_profile_summary": profile_outputs["profile_summary"],
        "cluster_profile_scaled_mean": profile_outputs["scaled_means"],
        "cluster_profile_interpretation":
profile_outputs["interpretation_text"],
        "analysis_text": analysis_text,
        "result_paths": result_paths,
        "figure_paths": figure_paths,
    }

def run_experiment(exp4_dir: str | Path | None = None) -> dict[str, Any]:
    warnings.filterwarnings("ignore", category=UserWarning)
    plt.rcParams["font.family"] = "DejaVu Sans"
    plt.rcParams["axes.unicode_minus"] = False

    paths = resolve_paths(exp4_dir)
    ensure_output_dirs(paths)

    print("Experiment 4: Wine Clustering Model Practice")
    print(f"Experiment 4 directory: {paths['exp4_dir']}")
    print(f>Data directory: {paths['data_dir']}")
    print("Original class labels will be removed before clustering.")

    # The workflow mirrors the report: data checks, model grids, comparison,
profiles.
    data_outputs = prepare_data(paths)
    data = data_outputs["data"]
    kmeans_outputs = run_kmeans_experiment(data, paths)
    agglomerative_outputs = run_agglomerative_experiment(data, paths)
    dbscan_outputs = run_dbscan_experiment(data, paths)

```



```

final_outputs = run_final_analysis(
    data,
    kmeans_outputs,
    agglomerative_outputs,
    dbscan_outputs,
    paths,
)

result_paths = [
    *data_outputs["result_paths"],
    *kmeans_outputs["result_paths"],
    *agglomerative_outputs["result_paths"],
    *dbscan_outputs["result_paths"],
    *final_outputs["result_paths"],
]

figure_paths = [
    *data_outputs["figure_paths"],
    *kmeans_outputs["figure_paths"],
    *agglomerative_outputs["figure_paths"],
    *dbscan_outputs["figure_paths"],
    *final_outputs["figure_paths"],
]

recommended = final_outputs["final_model_comparison"].iloc[0]
print("\nGenerated results files:")
for file_name in REQUIRED_RESULT_FILES:
    print(f"- {file_name}")
print("\nGenerated figures files:")
for file_name in REQUIRED_FIGURE_FILES:
    print(f"- {file_name}")
print(
    "\nRecommended model: "
    f"{recommended['model']} ({recommended['setting']})"
)
print("\nShort analysis text:")
print(final_outputs["analysis_text"])

print("\nExperiment 4 completed successfully.")
print(f"Results directory: {paths['results_dir']}")
print(f"Figures directory: {paths['figures_dir']}")

```

```

return {
    "paths": paths,
    "data": data,
    "kmeans": kmeans_outputs,
    "agglomerative": agglomerative_outputs,
    "dbscan": dbscan_outputs,
    "final": final_outputs,
    "result_paths": result_paths,
    "figure_paths": figure_paths,
}

def main() -> None:
    run_experiment()

if __name__ == "__main__":
    main()

```

4. Experimental Data and Results

Experimental Data:

🔄 🖥️ > 此电脑 > file (D:) > daima > cursor > 大数据分析 > 实验四 > wine				
📁 📄 🗑️ 🔍 🔄 排序 ⌵ 👁️ 查看 ⌵ ...				
名称	修改日期	类型	大小	
📄 Index	2023/5/22 22:22	文件	1 KB	
📄 wine.data	2023/5/22 22:22	DATA 文件	11 KB	
📄 wine.names	2023/5/22 22:22	NAMES 文件	3 KB	

```

... {
    'project_root': WindowsPath('D:/daima/cursor/大数据分析'),
    'exp4_dir': WindowsPath('D:/daima/cursor/大数据分析/实验四'),
    'data_dir': WindowsPath('D:/daima/cursor/大数据分析/实验四/data'),
    'results_dir': WindowsPath('D:/daima/cursor/大数据分析/实验四/results'),
    'figures_dir': WindowsPath('D:/daima/cursor/大数据分析/实验四/figures'),
    'wine_data': WindowsPath('D:/daima/cursor/大数据分析/实验四/data/wine.data'),
    'wine_names': WindowsPath('D:/daima/cursor/大数据分析/实验四/data/wine.names'),
    'wine_index': WindowsPath('D:/daima/cursor/大数据分析/实验四/data/Index')}

```

	class	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color Intensity	Hue	OD280/OD315 of diluted wines	Proline
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

```

... class in X columns: False
X shape: (178, 13)
y_true_optional shape: (178,)

```

	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color Intensity	Hue	OD280/OD315 of diluted wines	Proline
0	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

	item	value
0	dataset_file	D:\daima\cursor\大数据分析\实验四\data\wine.data
1	n_samples	178
2	n_original_columns	14
3	n_features_used_for_clustering	13
4	removed_label_column	class
5	feature_columns_used_for_clustering	Alcohol; Malic acid; Ash; Alcalinity of ash; M...
6	optional_original_class_distribution_note	Original class labels are optional information...
7	optional_original_class_1_count	59
8	optional_original_class_2_count	71
9	optional_original_class_3_count	48
10	dtype_class	int64
11	dtype_Alcohol	float64
12	dtype_Malic acid	float64
13	dtype_Ash	float64
14	dtype_Alcalinity of ash	float64
15	dtype_Magnesium	int64
16	dtype_Total phenols	float64
17	dtype_Flavanoids	float64
18	dtype_Nonflavanoid phenols	float64
19	dtype_Proanthocyanins	float64

...	item	value
0	total_missing_values_in_original_data	0
1	total_missing_values_in_X	0
2	duplicate_rows_in_original_data	0
3	duplicate_rows_in_X	0
4	missing_values_class	0
5	missing_values_Alcohol	0
6	missing_values_Malic acid	0
7	missing_values_Ash	0
8	missing_values_Alcalinity of ash	0
9	missing_values_Magnesium	0
10	missing_values_Total phenols	0
11	missing_values_Flavanoids	0
12	missing_values_Nonflavanoid phenols	0
13	missing_values_Proanthocyanins	0
14	missing_values_Color intensity	0
15	missing_values_Hue	0
16	missing_values_OD280/OD315 of diluted wines	0
17	missing_values_Proline	0

	data_version	feature	count	mean	std	min	25%	50%	75%	max
0	before_scaling	Alcohol	178.0	13.000618	0.811827	11.03	12.3625	13.050	13.6775	14.83
1	before_scaling	Malic acid	178.0	2.336348	1.117146	0.74	1.6025	1.865	3.0825	5.80
2	before_scaling	Ash	178.0	2.366517	0.274344	1.36	2.2100	2.360	2.5575	3.23
3	before_scaling	Alcalinity of ash	178.0	19.494944	3.339564	10.60	17.2000	19.500	21.5000	30.00
4	before_scaling	Magnesium	178.0	99.741573	14.282484	70.00	88.0000	98.000	107.0000	162.00

A	B
item	value
dataset_file	D:\daima\cursor\大数据分析\实验四\data\wine.data
n_samples	
n_original_columns	
n_features_used_for_clustering	
removed_label_column	class
feature_columns_used_for_clustering	Alcohol; Malic acid; Ash; Alcalinity of ash; Magnesium; Total phenols; Flavanoids; Nonflavanoid phenols; Proanthocyanins; Color intensity; Hue; OD280/OD315 of diluted wines
optional_original_class_distribution_note	Original class labels are optional information only and are not used for clustering training, parameter selection, or main evaluation.
optional_original_class_1_count	
optional_original_class_2_count	
optional_original_class_3_count	
dtype_class	int64
dtype_Alcohol	float64
dtype_Malic acid	float64
dtype_Ash	float64
dtype_Alcalinity of ash	float64
dtype_Magnesium	int64
dtype_Total phenols	float64
dtype_Flavanoids	float64
dtype_Nonflavanoid phenols	float64
dtype_Proanthocyanins	float64
dtype_Color intensity	float64
dtype_Hue	float64
dtype_OD280/OD315 of diluted wines	float64
dtype_Proline	int64

	A	B
1	item	value
2	total_missing_values_in_original_data	0
3	total_missing_values_in_X	0
4	duplicate_rows_in_original_data	0
5	duplicate_rows_in_X	0
6	missing_values_class	0
7	missing_values_Alcohol	0
8	missing_values_Malic acid	0
9	missing_values_Ash	0
10	missing_values_Alcalinity of ash	0
11	missing_values_Magnesium	0
12	missing_values_Total phenols	0
13	missing_values_Flavanoids	0
14	missing_values_Nonflavanoid phenols	0
15	missing_values_Proanthocyanins	0
16	missing_values_Color intensity	0
17	missing_values_Hue	0
18	missing_values_OD280/OD315 of diluted wines	0
19	missing_values_Proline	0

	A	B	C	D	E	F	G	H	I	J
1	data version	feature	count	mean	std	min	25%	50%	75%	max
2	before_scaling	Alcohol	178	13.00061797753909	0.8118265380058575	11.03	12.362499999999999	13.05	13.6775	
3	before_scaling	Malic acid	178	2.363483146067416	1.1171460976144627	0.74	1.6025	1.8650000000000002	3.0825	
4	before_scaling	Ash	178	2.3665168539328845	0.27434400906081485	1.36	2.21	2.36	2.5575	
5	before_scaling	Alcalinity of ash	178	19.49494382022472	3.339563767173505	10.6	17.2	19.5	21.5	
6	before_scaling	Magnesium	178	99.74157303370787	14.282483515295665	70	88	98	107	
7	before_scaling	Total phenols	178	2.295112359550562	0.6258510488339893	0.98	1.7425	2.355	2.8	
8	before_scaling	Flavanoids	178	2.0292666629213487	0.9988586850169467	0.34	1.205	2.135	2.875	
9	before_scaling	Nonflavanoid phenols	178	0.3618539325842696	0.12445334029667937	0.13	0.27	0.34	0.4375	
10	before_scaling	Proanthocyanins	178	1.5908989764044945	0.5723589626747613	0.41	1.25	1.5550000000000002	1.95	
11	before_scaling	Color intensity	178	5.08308982022472	2.318285871822413	1.28	3.2199999999999998	4.6599999999999995	6.2	
12	before_scaling	Hue	178	0.9574494382022471	0.22857156582982338	0.48	0.7825	0.965	1.12	
13	before_scaling	OD280/OD315 of diluted wines	178	2.6116853932584267	0.7099904287650504	1.27	1.9375	2.78	3.17	
14	before_scaling	Proline	178	746.8932584269663	314.9074742768491	278	500.5	673.5	985	
15	after_scaling	Alcohol	178	-8.38E-16	1.0028208800748637	-2.434235347	-0.788244781	0.060999882261548254	0.8361286478137232	2.25977152000317
16	after_scaling	Malic acid	178	-1.20E-16	1.002820880074864	-1.43298305	-0.658748586	-0.423112011	0.6697928690018488	3.10919246715890
17	after_scaling	Ash	178	-8.37E-16	1.002820880074864	-3.679162234	-0.572122459	-0.023821323	0.6981085071784328	3.1563252699904
18	after_scaling	Alcalinity of ash	178	-3.99E-17	1.0028208800748637	-2.671018136	-0.689137187	0.00151829490483276	0.6020882794831586	3.1545107139410
19	after_scaling	Magnesium	178	-3.99E-17	1.0028208800748637	-2.088255198	-0.824415067	-0.122251661	0.5096384050685507	4.37137213955478
20	after_scaling	Total phenols	178	0.1	0.0028208800748637	-2.107246023	-0.855468218	0.00595885564457041	0.8089973946320397	2.53951546617814
21	after_scaling	Flavanoids	178	-3.99E-16	1.0028208800748637	-1.695970527	-0.827539312	0.10614973996849836	0.8490851144982885	3.06283173728235
22	after_scaling	Nonflavanoid phenols	178	3.5926318100229784E-16	1.0028208800748637	-1.868234024	-0.740141175	-0.17609475	0.609541340708147	2.40240318985375
23	after_scaling	Proanthocyanins	178	-1.20E-16	1.0028208800748635	-2.069034181	-0.597283511	-0.062897852	0.6291753797906147	3.4850725120254
24	after_scaling	Color intensity	178	2.494883201404846E-17	1.0028208800748637	-1.634288276	-0.795102509	-0.159224634	0.4939560402774766	3.4354319244178
25	after_scaling	Hue	178	1.995905611238769E-16	1.0028208800748637	-2.094732405	-0.767562444	0.03312687210061617	0.7131643730351397	3.30169421530203
26	after_scaling	OD280/OD315 of diluted wines	178	3.193450497798203E-16	1.0028208800748637	-1.895053894	-0.952248315	0.2377347570102168	0.7885874549958394	1.9609149917344
27	after_scaling	Proline	178	-1.60E-16	1.0028208800748637	-1.493187646	-0.784637789	-0.233720372	0.7882494276592087	2.9714275599038

Experimental Results:

	count	mean	std	min	25%	50%	75%	max
Alcohol	178.0	-8.382808e-16	1.002821	-2.434235	-0.788245	0.061000	0.836129	2.259772
Malic acid	178.0	-1.197544e-16	1.002821	-1.432983	-0.658749	-0.423112	0.669793	3.109192
Ash	178.0	-8.370333e-16	1.002821	-3.679162	-0.572122	-0.023821	0.698109	3.156325
Alcalinity of ash	178.0	-3.991813e-17	1.002821	-2.671018	-0.689137	0.001518	0.602088	3.154511
Magnesium	178.0	-3.991813e-17	1.002821	-2.088255	-0.824415	-0.122282	0.509638	4.371372

Explained variance ratio: [0.36198848 0.1920749]		
	PC1	PC2
0	3.316751	1.443463
1	2.209465	-0.333393
2	2.516740	1.031151
3	3.757066	2.756372
4	1.008908	0.869831

	model	n_clusters	inertia	silhouette_score	calinski_harabasz_score	davies_bouldin_score	training_time_seconds	cluster_size_summary
0	KMeans	2	1658.758852	0.259317	69.523332	1.525997	7.836127	0:87; 1:91
1	KMeans	3	1277.928489	0.284859	70.940008	1.389188	0.240155	0:65; 1:51; 2:62
2	KMeans	4	1175.351881	0.258608	56.188782	1.816991	0.753099	0:55; 1:49; 2:45; 3:29
3	KMeans	5	1107.007343	0.231528	47.156356	1.681074	1.633017	0:26; 1:55; 2:48; 3:26; 4:23
4	KMeans	6	1044.540748	0.191923	41.807271	1.821907	1.316939	0:55; 1:24; 2:23; 3:20; 4:28; 5:28
5	KMeans	7	981.595233	0.203628	38.685534	1.660585	0.575593	0:23; 1:54; 2:18; 3:20; 4:3; 5:28; 6:32
6	KMeans	8	935.201211	0.157014	35.805250	1.740859	0.770658	0:19; 1:39; 2:23; 3:30; 4:23; 5:3; 6:21; 7:20
7	KMeans	9	885.605913	0.150398	34.072520	1.680480	0.494981	0:21; 1:31; 2:26; 3:28; 4:18; 5:4; 6:29; 7:3; ...
8	KMeans	10	845.895237	0.143638	32.397182	1.706970	0.576071	0:4; 1:10; 2:18; 3:19; 4:17; 5:27; 6:24; 7:22; ...

	model	linkage	n_clusters	silhouette_score	calinski_harabasz_score	davies_bouldin_score	training_time_seconds	cluster_size_summary
0	AgglomerativeClustering	ward	2	0.267013	65.360838	1.411757	0.312704	0:122; 1:56
1	AgglomerativeClustering	ward	3	0.277444	67.647468	1.418592	0.001878	0:58; 1:56; 2:64
2	AgglomerativeClustering	ward	4	0.225837	51.464146	1.788651	0.002457	0:56; 1:30; 2:64; 3:28
3	AgglomerativeClustering	ward	5	0.186742	43.679272	1.922855	0.001929	0:64; 1:30; 2:36; 3:28; 4:20
4	AgglomerativeClustering	ward	6	0.179666	39.128964	1.812288	0.003475	0:30; 1:58; 2:36; 3:28; 4:20; 5:6

	model	eps	min_samples	cluster_count_excluding_noise	noise_count	noise_ratio	silhouette_score	calinski_harabasz_score	davies_bouldin_score	training_time_seconds
0	DBSCAN	0.3	3	0	178	1.0	NaN	NaN	NaN	0.012361
1	DBSCAN	0.3	4	0	178	1.0	NaN	NaN	NaN	0.007068
2	DBSCAN	0.3	5	0	178	1.0	NaN	NaN	NaN	0.006401
3	DBSCAN	0.3	6	0	178	1.0	NaN	NaN	NaN	0.008374
4	DBSCAN	0.3	8	0	178	1.0	NaN	NaN	NaN	0.005762

	model	setting	n_clusters	noise_ratio	silhouette_score	calinski_harabasz_score	davies_bouldin_score	training_time_seconds	notes
0	DBSCAN	eps=2.30, min_samples=10	2	0.359551	0.364113	64.741430	1.048405	0.004770	Best valid DBSCAN setting; noise ratio is cons...
1	KMeans	k=3	3	0.000000	0.284859	70.940008	1.389188	0.240155	Fixed k=3 retained for cluster interpretation.
2	KMeans	best by silhouette, k=3	3	0.000000	0.284859	70.940008	1.389188	0.240155	Selected by internal clustering metrics only.
3	AgglomerativeClustering	linkage=ward, k=3	3	0.000000	0.277444	67.647468	1.418592	0.001878	Ward linkage with k=3 retained as a hierarchic...
4	AgglomerativeClustering	best by silhouette, linkage=ward, k=3	3	0.000000	0.277444	67.647468	1.418592	0.001878	Selected by internal clustering metrics only.

cluster	sample_count	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines
0	0	65	12.250923	1.897385	2.231231	20.063077	92.738462	2.247692	2.050000	0.357692	1.624154	2.973077	1.062708
1	1	51	13.134118	3.307255	2.417647	21.241176	98.666667	1.683922	0.818824	0.451961	1.145882	7.234706	0.691961
2	2	62	13.676774	1.997903	2.466290	17.462903	107.967742	2.847581	3.003226	0.292097	1.922097	5.453548	1.065484
0	0	-0.926072	-0.394042	-0.494517	0.170602	-0.491712	-0.075983	0.020813	-0.033534	0.058266	-0.901914	0.461804	0.270764
1	1	0.164907	0.871547	0.186898	0.524367	-0.075473	-0.979330	-1.215248	0.726064	-0.779706	0.941539	-1.164789	-1.292412
2	2	0.835232	-0.303810	0.364706	-0.610191	0.577587	0.885237	0.977820	-0.562090	0.580287	0.171063	0.473984	0.779247
Cluster profile interpretation based on KMeans k=3. The values are interpreted from standardized feature means; positive values are above the overall average and negative values are below the overall average. Cluster 0 contains 65 samples. It is relatively high in Hue (+0.46), OD280/OD315 of diluted wines (+0.27), Alcalinity of ash (+0.17), and relatively low in Alcohol (-0.93). Cluster 1 contains 51 samples. It is relatively high in Color intensity (+0.94), Malic acid (+0.87), Nonflavanoid phenols (+0.73), and relatively low in OD280/OD315 of diluted wines (-1.29). Cluster 2 contains 62 samples. It is relatively high in Proline (+1.13), Flavanoids (+0.98), Total phenols (+0.89), and relatively low in Alcalinity of ash (-0.61).													
cluster	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
0	0	-0.926072	-0.394042	-0.494517	0.170602	-0.491712	-0.075983	0.020813	-0.033534	0.058266	-0.901914	0.461804	0.270764
1	1	0.164907	0.871547	0.186898	0.524367	-0.075473	-0.979330	-1.215248	0.726064	-0.779706	0.941539	-1.164789	-1.292412
2	2	0.835232	-0.303810	0.364706	-0.610191	0.577587	0.885237	0.977820	-0.562090	0.580287	0.171063	0.473984	0.779247

	model	adjusted_rand_index_optional	normalized_mutual_info_optional	homogeneity_optional	completeness_optional	v_measure_optional	note
0	KMeans k=3	0.897495	0.875894	0.878843	0.872964	0.875894	Optional external comparison only; original la...
1	Best KMeans	0.897495	0.875894	0.878843	0.872964	0.875894	Optional external comparison only; original la...
2	Agglomerative ward k=3	0.789933	0.786465	0.790429	0.782541	0.786465	Optional external comparison only; original la...
3	Best Agglomerative	0.789933	0.786465	0.790429	0.782541	0.786465	Optional external comparison only; original la...
4	Best DBSCAN	0.282144	0.377302	0.366225	0.389070	0.377302	Optional external comparison only; original la...

Recommended model: DBSCAN (eps=2.30, min_samples=10). It has Silhouette=0.3641, Calinski-Harabasz=64.7414, and Davies-Bouldin=1.0484. The recommendation is based on the best performance across all metrics.

Generated results directory: D:\daima\cursor\大数据分析\实验四\results

Generated figures directory: D:\daima\cursor\大数据分析\实验四\figures

#	A	B	C	D	E	F	G	H
1	model	n_clusters	inertia	silhouette_score	calinski_harabasz_score	davies_bouldin_score	training_time_seconds	cluster_size_summary
2	KMeans	2	1658.7588504290952	0.299169553182343	69.52353173902912	1.5259965360652092	1.836156900001545	0:87; 1:91
3	KMeans	3	1277.928488844642	0.284859191899897	70.94000800315118	1.3891870771815449	0.24015520000102497	0:45; 1:51; 2:62
4	KMeans	4	1175.3518813658281	0.2589075384934024	56.18878220881203	1.8169905232251697	0.7539860600019766	0:55; 1:49; 2:45; 3:29
5	KMeans	5	1107.0073434603985	0.23152534284691684	47.15635601129616	1.6810742019980314	1.6330166999978246	0:26; 1:55; 2:48; 3:26; 4:23
6	KMeans	6	1044.5407477360443	0.19192316734484377	41.80727115962674	1.8219069855023633	1.31693860000008672	0:55; 1:24; 2:23; 3:20; 4:28; 5:28
7	KMeans	7	981.5952326111658	0.2036275812271175	38.68534127811424	1.6605851121157633	0.5755933000000368	0:23; 1:54; 2:18; 3:20; 4:3; 5:28; 6:32
8	KMeans	8	935.2012114738545	0.1570139556393107	35.80525027642259	1.7408592639953422	0.7706580000012764	0:19; 1:39; 2:23; 3:30; 4:23; 5:3; 6:21; 7:20
9	KMeans	9	885.6059127776966	0.15039835309243121	34.07251990665694	1.6804802172803202	0.49498099999982514	0:21; 1:31; 2:26; 3:28; 4:18; 5:4; 6:29; 7:3; 8:18
10	KMeans	10	845.8952366525514	0.1436384079124149	32.397182371661394	1.7069697874648369	0.5760706000000937	0:4; 1:10; 2:18; 3:19; 4:17; 5:27; 6:24; 7:22; 8:18; 9:19
11								

W365

企业工作台

kmeans_selection_results.csv

sglogglomerative_grid_results.csv

dsbcann_grid_search_results.csv

W365 企业工作台 s_kmeans_selection_results.cs... s_agglomerative_grid_results.cs... d_bscan_grid_search_results.cs... 开始 插入 页面 公式 数据 审阅 视图 工具 会员专享 删除重复项 有修改 分享

格式刷 粘贴 字体 B I U 背景 颜色 边框 表格 行和列 表格样式 条件格式 数据有效性 填充 求和 排序 筛选 冻结 查找

A1	fx model									
#	A	B	C	D	E	F	G	H	I	J
	model	eps	min_samples	cluster_count_excluding_noise	noise_count	noise_ratio	silhouette_score	calinski_harabasz_score	davies_bouldin_score	training_time_seconds
2	DBSCAN	0.3	3	0	178	1			0.01236069999680502	
3	DBSCAN	0.3	4	0	178	1			0.00706839999838846	
4	DBSCAN	0.3	5	0	178	1			0.006401200000254903	
5	DBSCAN	0.3	6	0	178	1			0.00837390000013148	
6	DBSCAN	0.3	8	0	178	1			0.0057617000020400155	
7	DBSCAN	0.3	10	0	178	1			0.00960799999666051	
8	DBSCAN	0.4	3	0	178	1			0.006010900000546826	
9	DBSCAN	0.4	4	0	178	1			0.00507550000112232	
10	DBSCAN	0.4	5	0	178	1			0.003547200002183672	
11	DBSCAN	0.4	6	0	178	1			0.00472069999870259	
12	DBSCAN	0.4	8	0	178	1			0.00367579999758801	
13	DBSCAN	0.4	10	0	178	1			0.003107299999508541	
14	DBSCAN	0.5	3	0	178	1			0.003950099999201484	
15	DBSCAN	0.5	4	0	178	1			0.0026012000016635284	
16	DBSCAN	0.5	5	0	178	1			0.0021381000005931128	
17	DBSCAN	0.5	6	0	178	1			0.0038079999980691355	
18	DBSCAN	0.5	8	0	178	1			0.004636699999537086	
19	DBSCAN	0.5	10	0	178	1			0.002333999998787884	
20	DBSCAN	0.6	3	0	178	1			0.006567799999174895	
21	DBSCAN	0.6	4	0	178	1			0.003412499998376065	
22	DBSCAN	0.6	5	0	178	1			0.0035767999979725373	
23	DBSCAN	0.6	6	0	178	1			0.00311879999935627	
24	DBSCAN	0.6	8	0	178	1			0.004062400003022049	
25	DBSCAN	0.6	10	0	178	1			0.004450799999176325	
26	DBSCAN	0.7	3	0	178	1			0.003808699998338241	
27	DBSCAN	0.7	4	0	178	1			0.004717999999128315	
28	DBSCAN	0.7	5	0	178	1			0.00353799999815971	
29	DBSCAN	0.7	6	0	178	1			0.00336649999776273	

< > d_bscan_grid_search_results +

[illegible][illegible]

#	A	B	C	D	E	F	G	H	I	J	
1	cluster	sample_count	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proant
2	0	65	12.03923076923076	1.8973846153846154	2.2312307692307694	20.06307692307692	92.73846153846154	2.2476923076923074	0.8188235294117646	0.3576923076923077	1.6241
3	1	51	13.13417647058824	3.307254901960784	2.4176470588235293	21.241176470588233	98.66666666666667	1.683921568627451	0.4519607843137255	0.4519607843137255	1.1458
4	2	62	13.67677419354838	1.9979032225806457	2.466290322380645	17.462903223806453	107.96774193548387	2.8475806451612904	3.0032238064516126	0.2920967741935484	1.9220

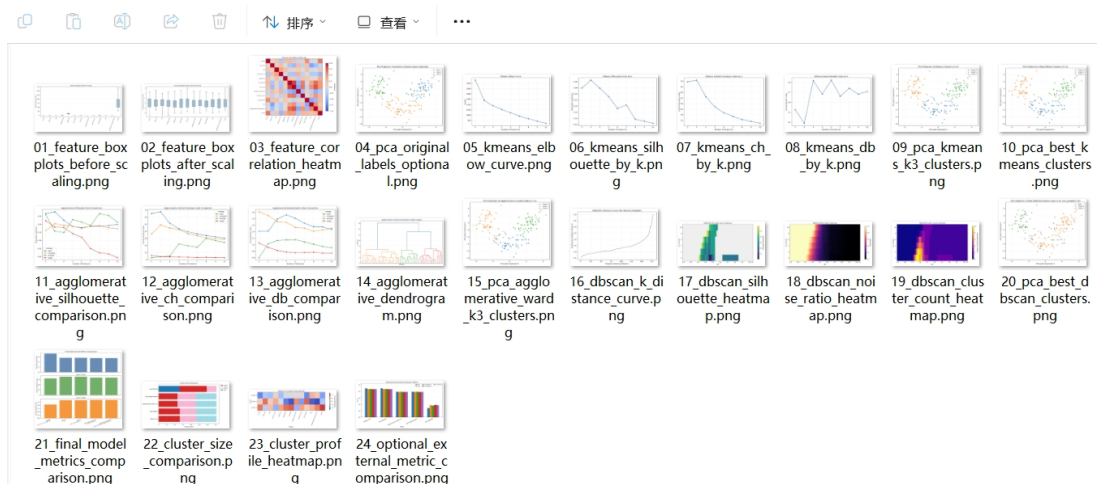
#	model	adjusted_rand_index_optional	normalized_mutual_info_optional	homogeneity_optional	completeness_optional	v_measure_optional	note
2	KMeans k=3	0.8974949815093207	0.8758935341223069	0.8788432003662366	0.8729636016078731	0.875893534122307	Optional external comparison only; os
3	Best KMeans	0.8974949815093207	0.8758935341223069	0.8788432003662366	0.8729636016078731	0.875893534122307	Optional external comparison only; os
4	Agglomerative ward k=3	0.7899332213582837	0.7864652657004837	0.790429271831654	0.7825408201875712	0.7864652657004838	Optional external comparison only; os
5	Best Agglomerative	0.7899332213582837	0.7864652657004837	0.790429271831654	0.7825408201875712	0.7864652657004838	Optional external comparison only; os
6	Best DBSCAN	0.28214436597967496	0.3773020810877838	0.366225493589338105	0.38906959415334064	0.3773020810877838	Optional external comparison only; os

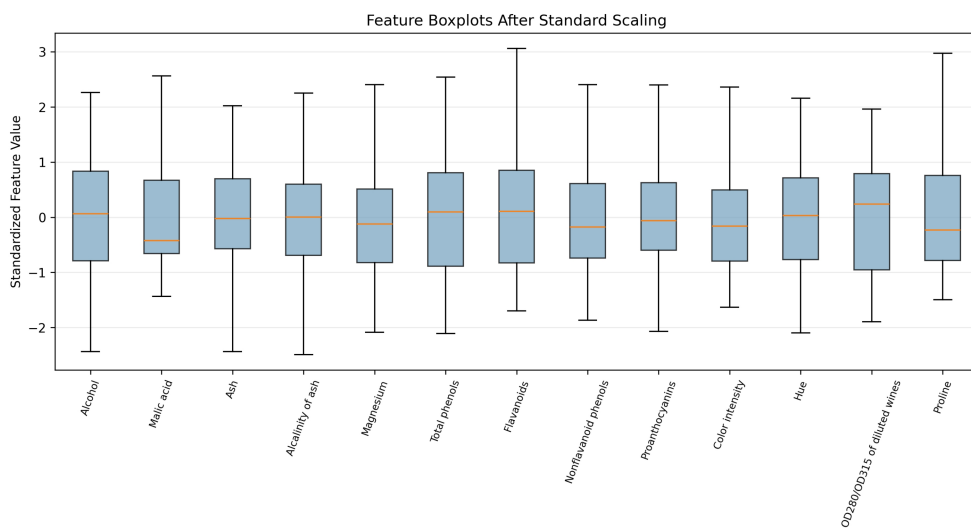
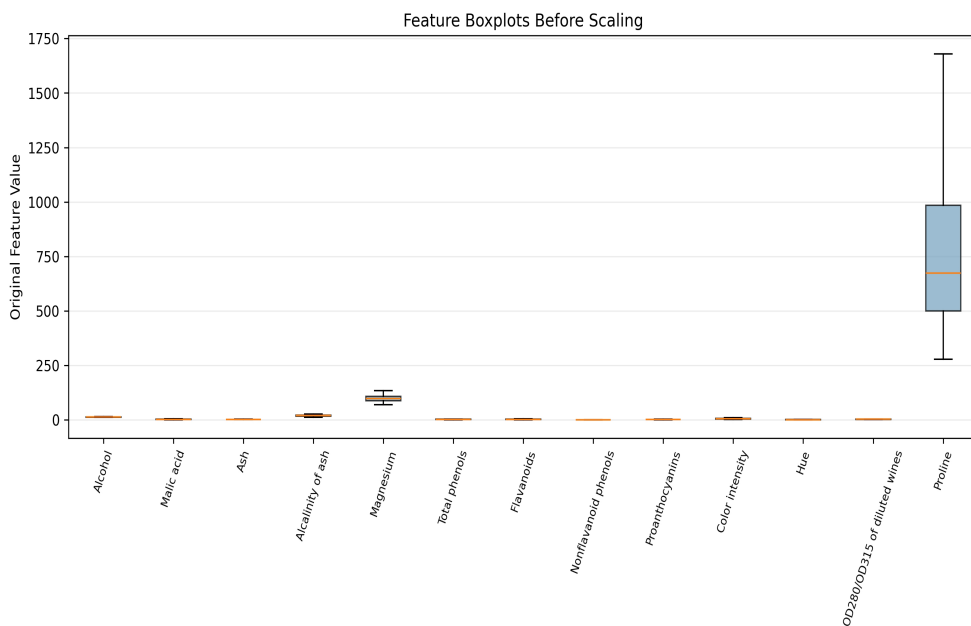
Cluster profile interpretation based on **KMeans k=3**.
The values are interpreted from standardized feature means; positive values are above the overall average and negative values are below the overall average.
Cluster 0 contains 65 samples. It is relatively high in Hue (+0.46), OD280/OD315 of diluted wines (+0.27), **Alcalinity** of ash (+0.17), and relatively low in Alcohol (-0.93), Color intensity (-0.98), Proline (-0.75).
Cluster 1 contains 51 samples. It is relatively high in Color intensity (+0.94), Malic acid (+0.87), **Nonflavanoid** phenols (+0.73), and relatively low in OD280/OD315 of diluted wines (-1.29), **Flavanoids** (-1.22), Hue (-1.16).
Cluster 2 contains 62 samples. It is relatively high in Proline (+1.13), **Flavanoids** (+0.98), Total phenols (+0.89), and relatively low in **Alcalinity** of ash (-0.61), **Nonflavanoid** phenols (-0.56), Malic acid (-0.30).]

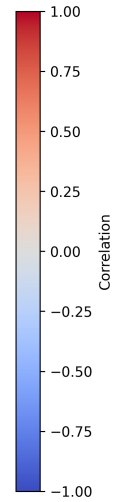
此电脑 > file (D:) > daima > cursor > 大数据分析 > 实验四 > results

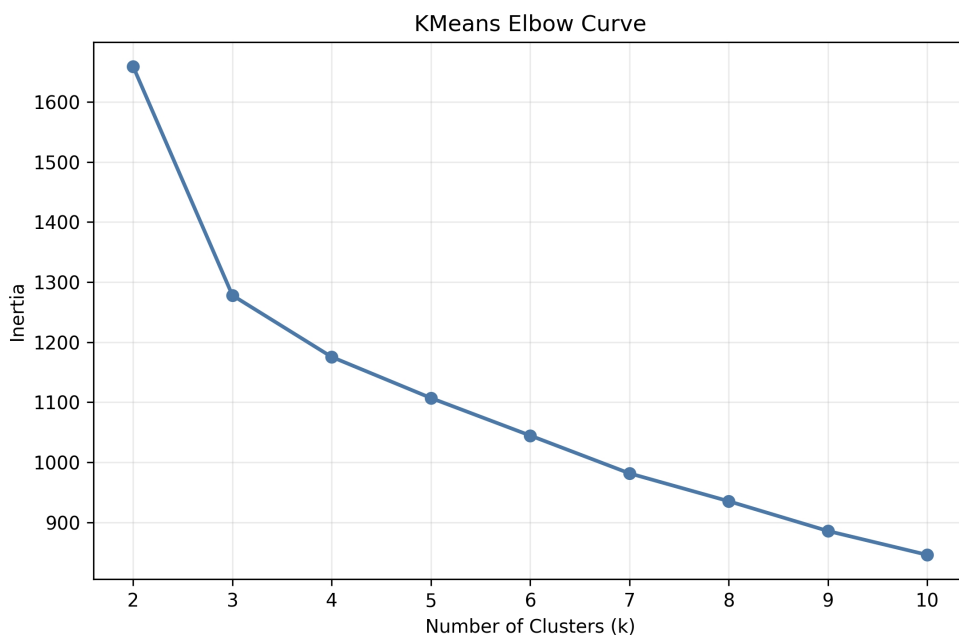
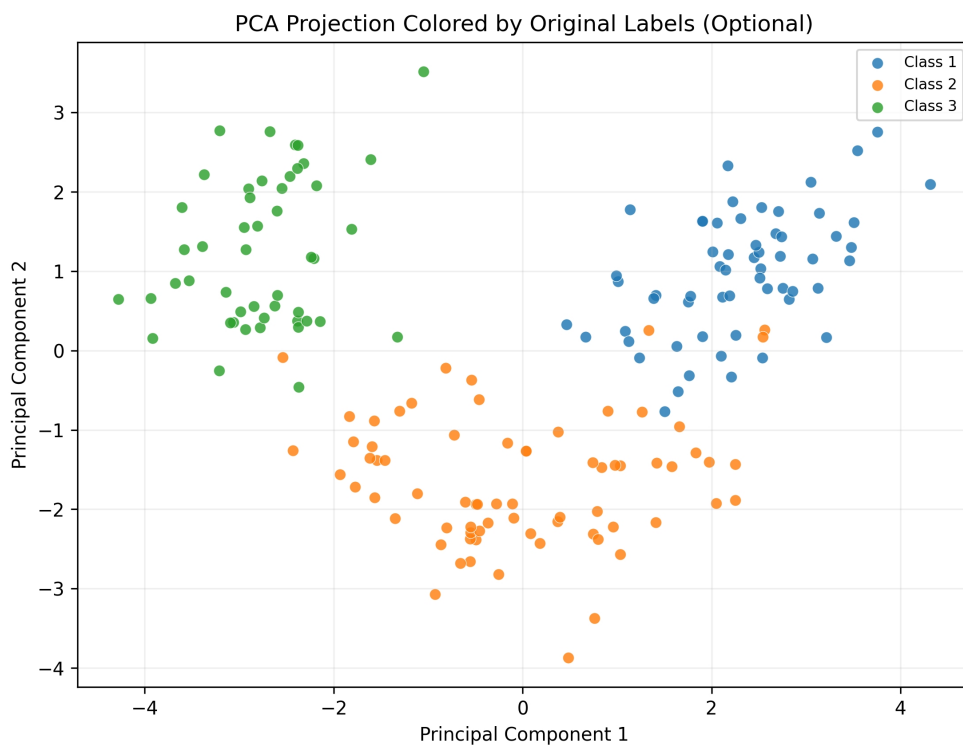
名称	修改日期	类型	大小
agglomerative_grid_results.csv	2026/5/29 21:27	XLS 工作表	6 KB
cluster_profile_interpretation.txt	2026/5/29 21:28	Text Document	1 KB
cluster_profile_scaled_mean.csv	2026/5/29 21:28	XLS 工作表	1 KB
cluster_profile_summary.csv	2026/5/29 21:28	XLS 工作表	1 KB
data_basic_info.csv	2026/5/29 21:27	XLS 工作表	2 KB
dbscan_grid_search_results.csv	2026/5/29 21:27	XLS 工作表	19 KB
feature_descriptive_statistics.csv	2026/5/29 21:27	XLS 工作表	4 KB
final_cluster_assignments.csv	2026/5/29 21:28	XLS 工作表	3 KB
final_model_comparison.csv	2026/5/29 21:28	XLS 工作表	2 KB
kmeans_k_selection_results.csv	2026/5/29 21:27	XLS 工作表	2 KB
missing_duplicate_check.csv	2026/5/29 21:27	XLS 工作表	1 KB
model_ranking_summary.csv	2026/5/29 21:28	XLS 工作表	2 KB
optional_agglomerative_class_cross...	2026/5/29 21:28	XLS 工作表	1 KB
optional_dbscan_class_crosstab.csv	2026/5/29 21:28	XLS 工作表	1 KB
optional_external_label_comparison...	2026/5/29 21:28	XLS 工作表	2 KB
optional_kmeans_class_crosstab.csv	2026/5/29 21:28	XLS 工作表	1 KB

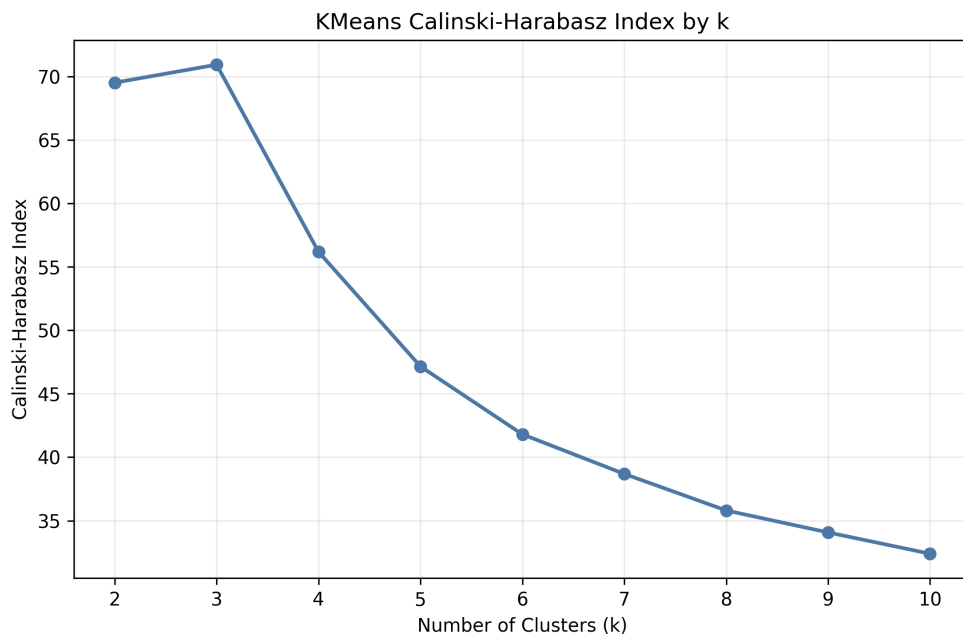
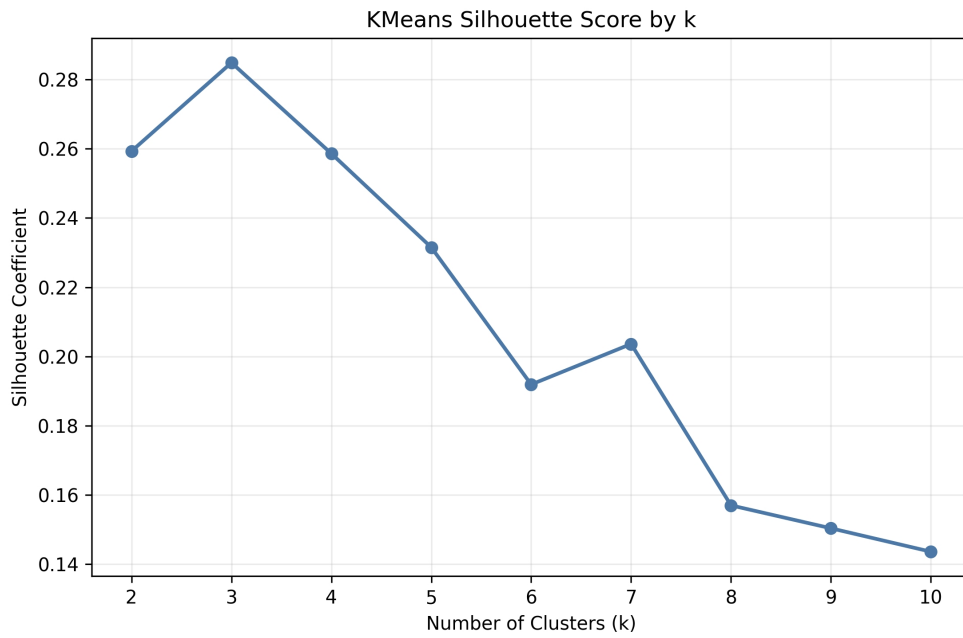
此电脑 > file (D:) > daima > cursor > 大数据分析 > 实验四 > figures

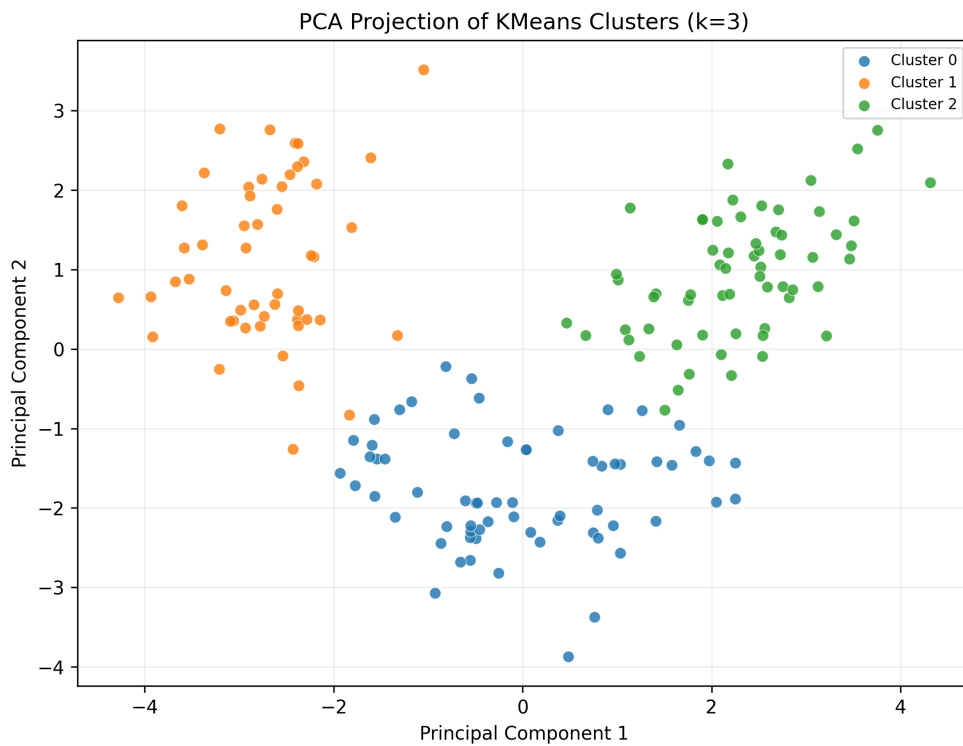
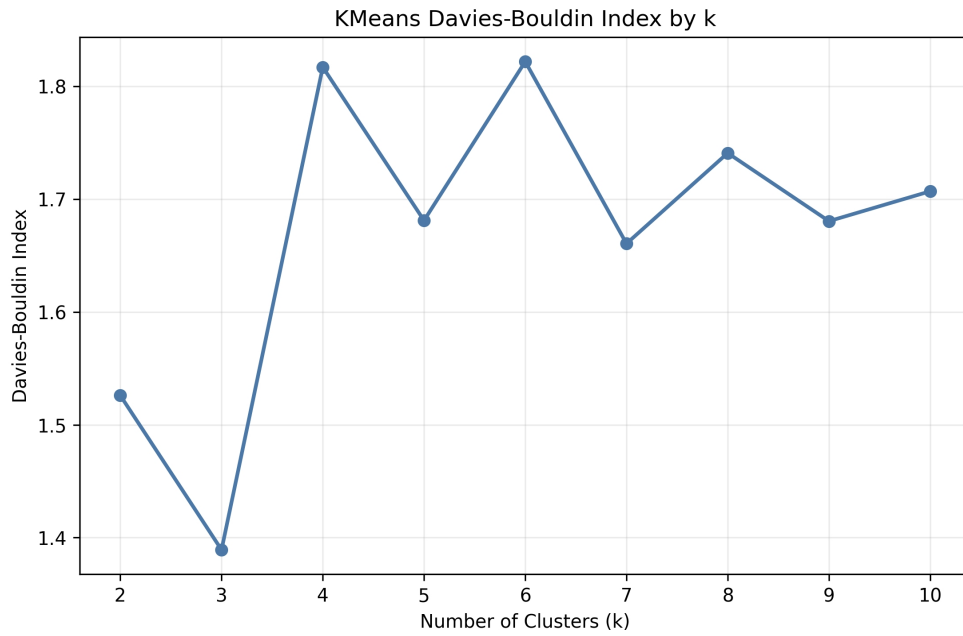


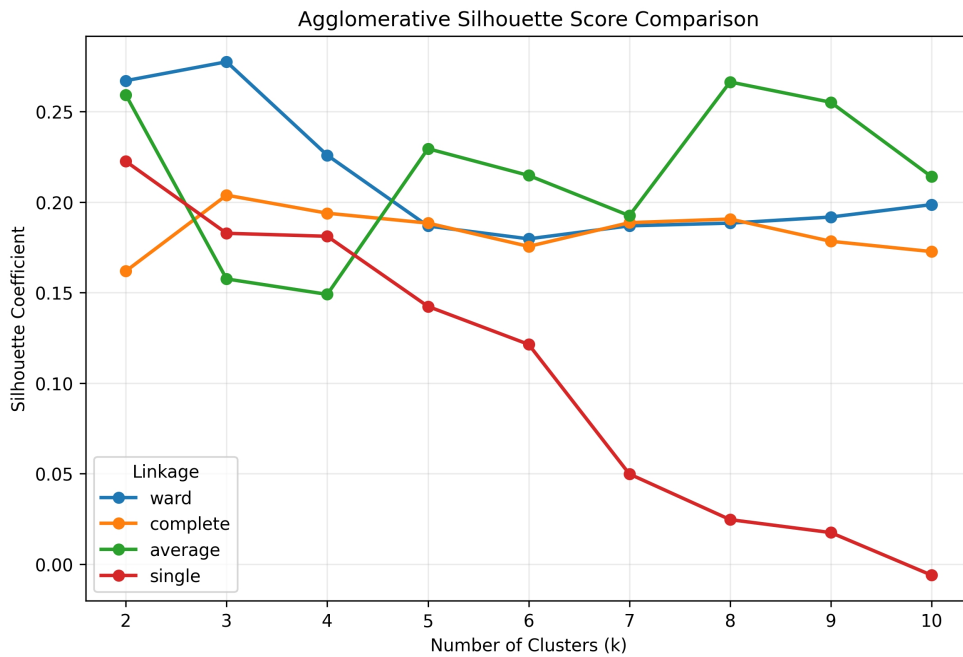
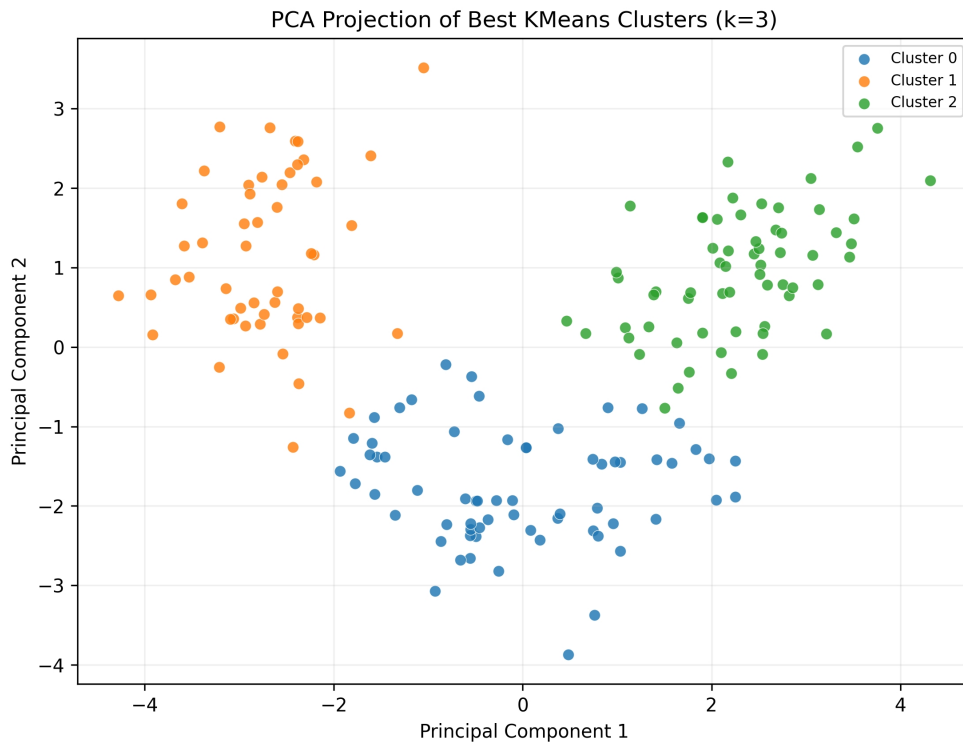


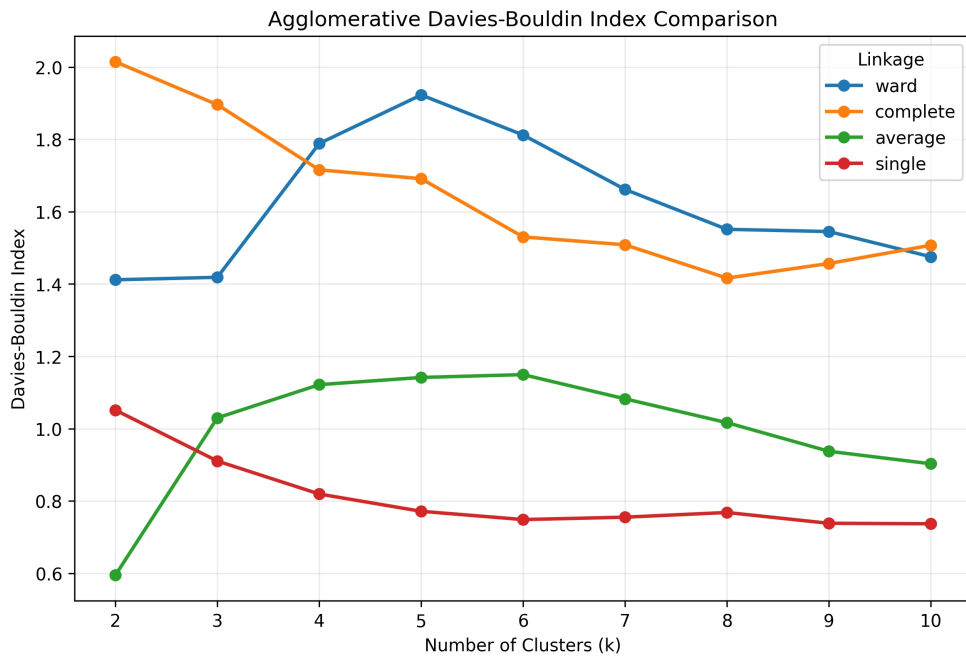
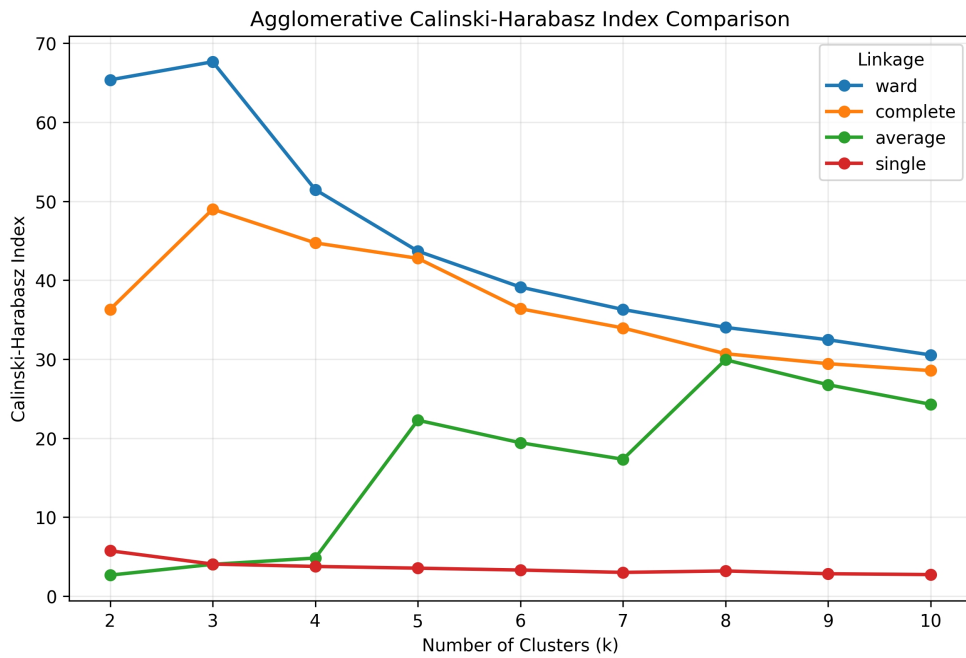
[illegible]

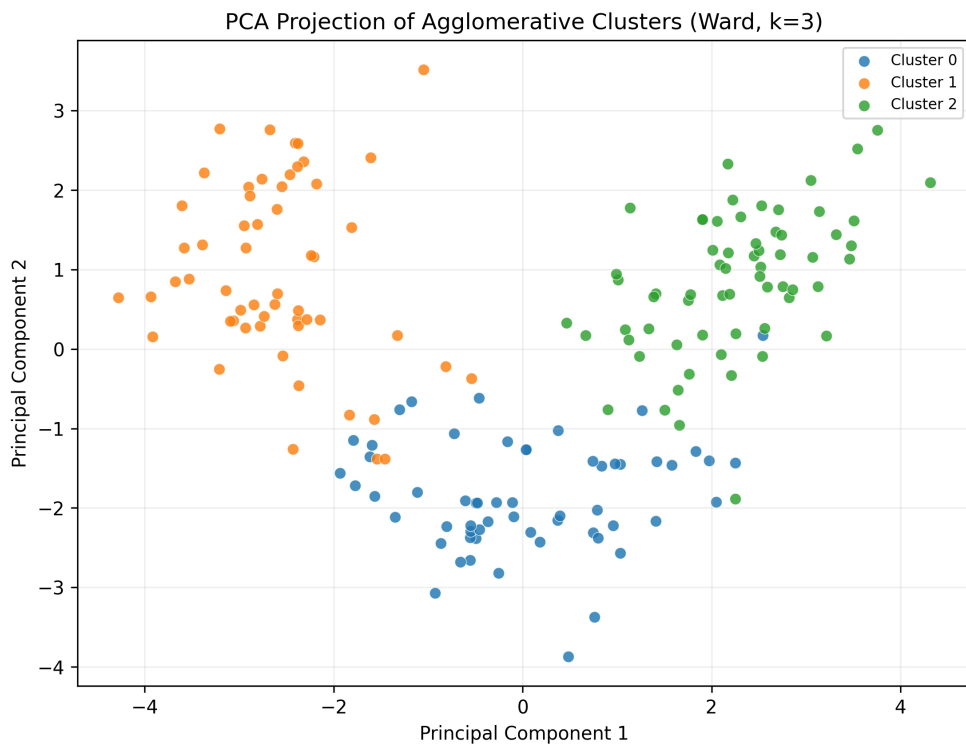
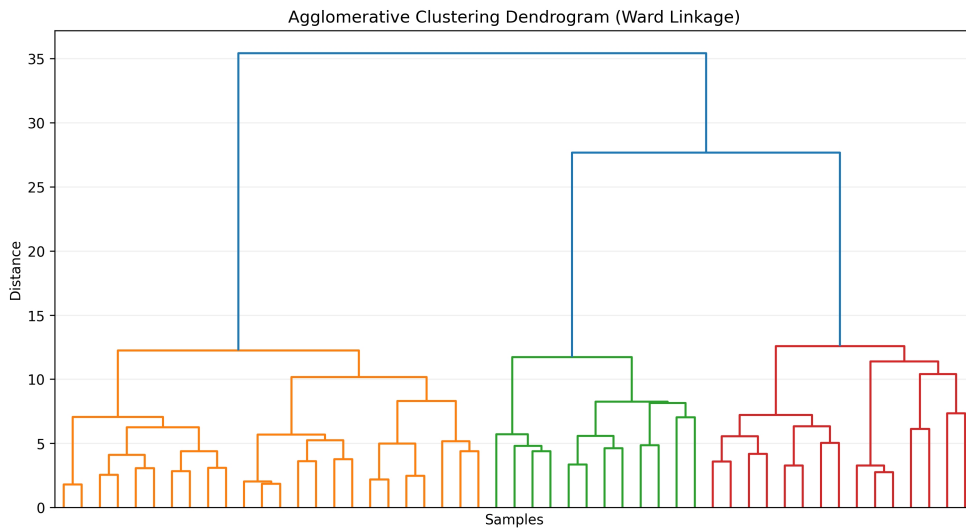


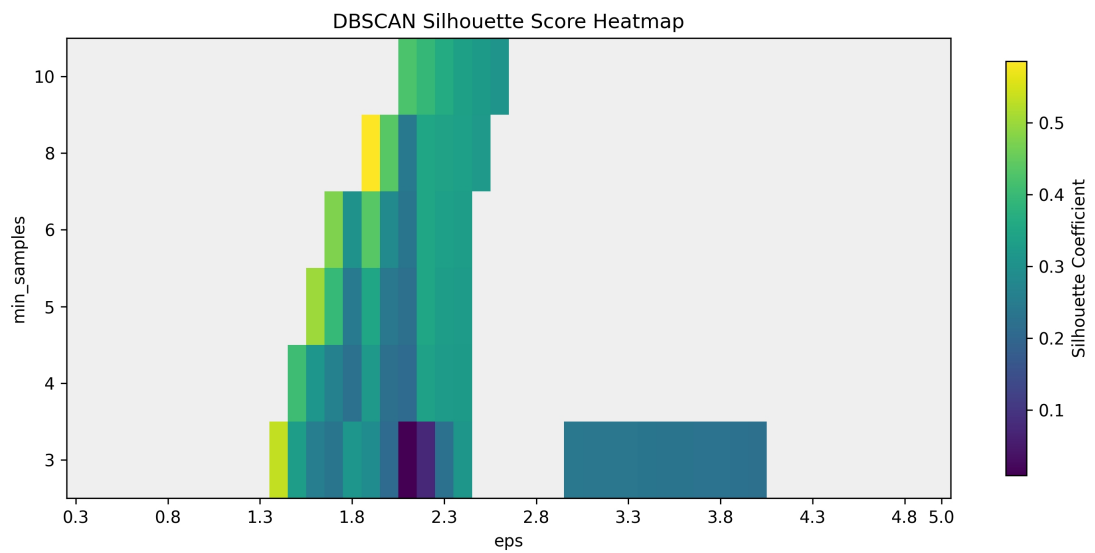
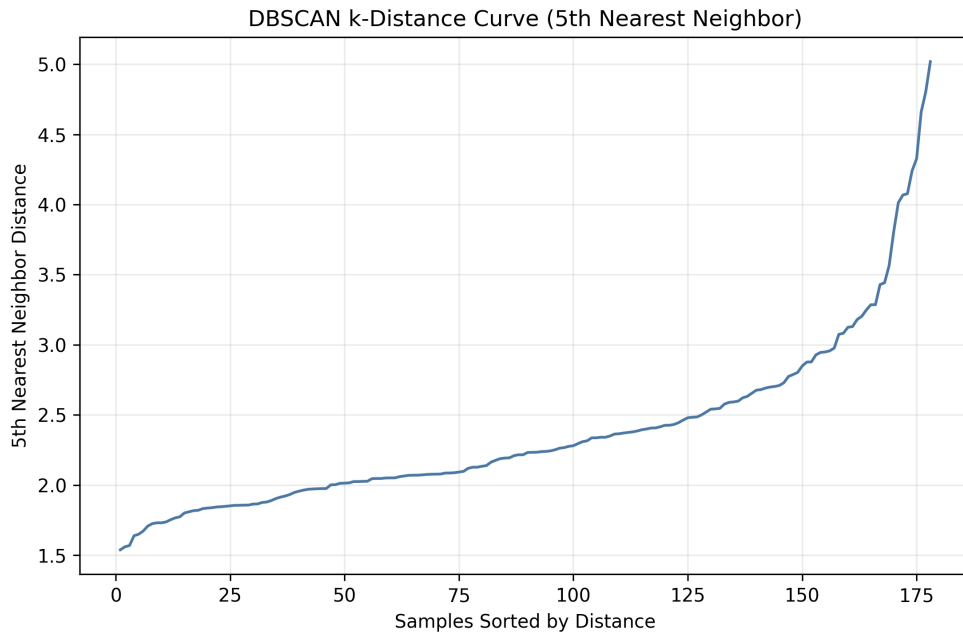


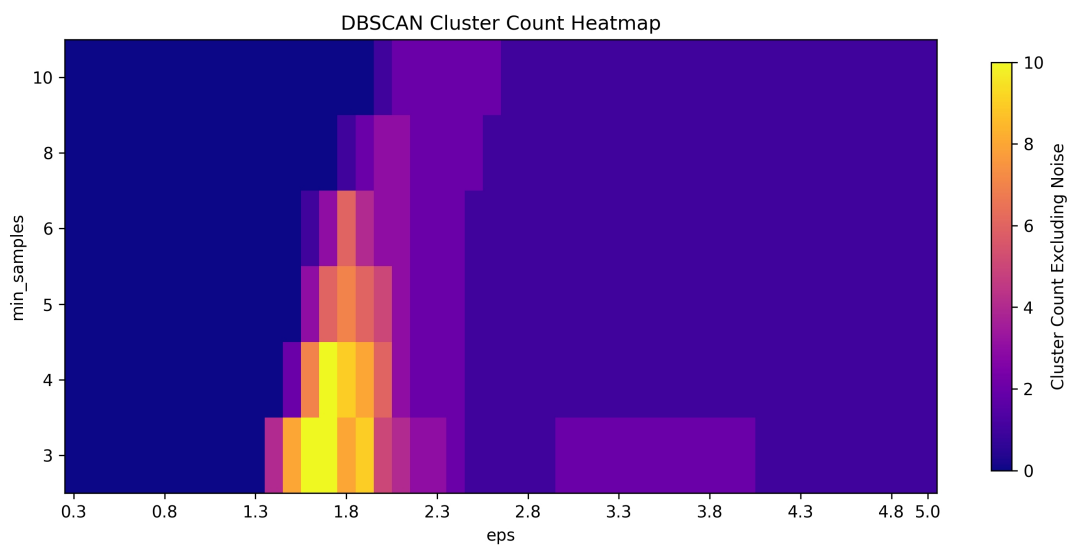
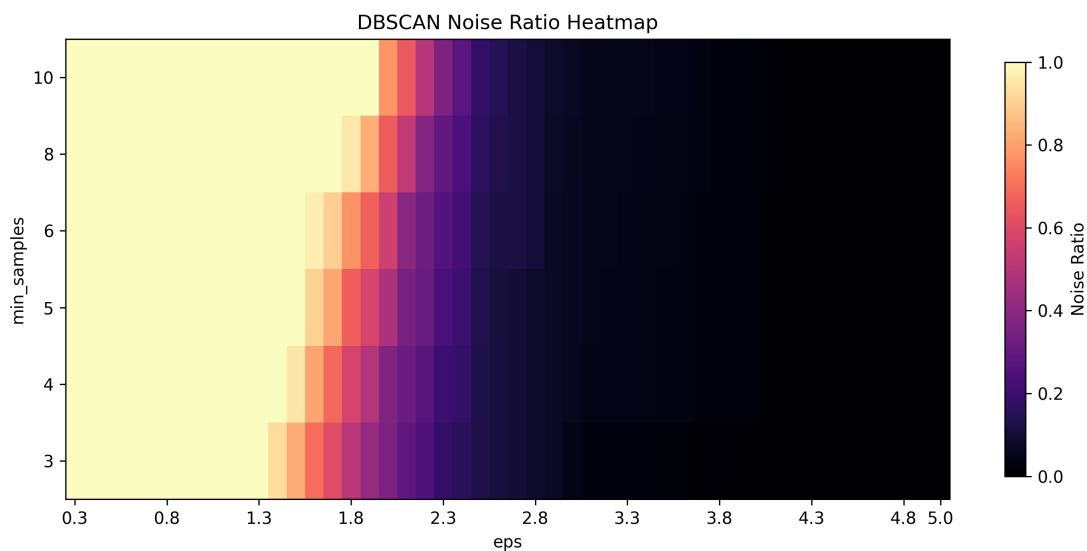




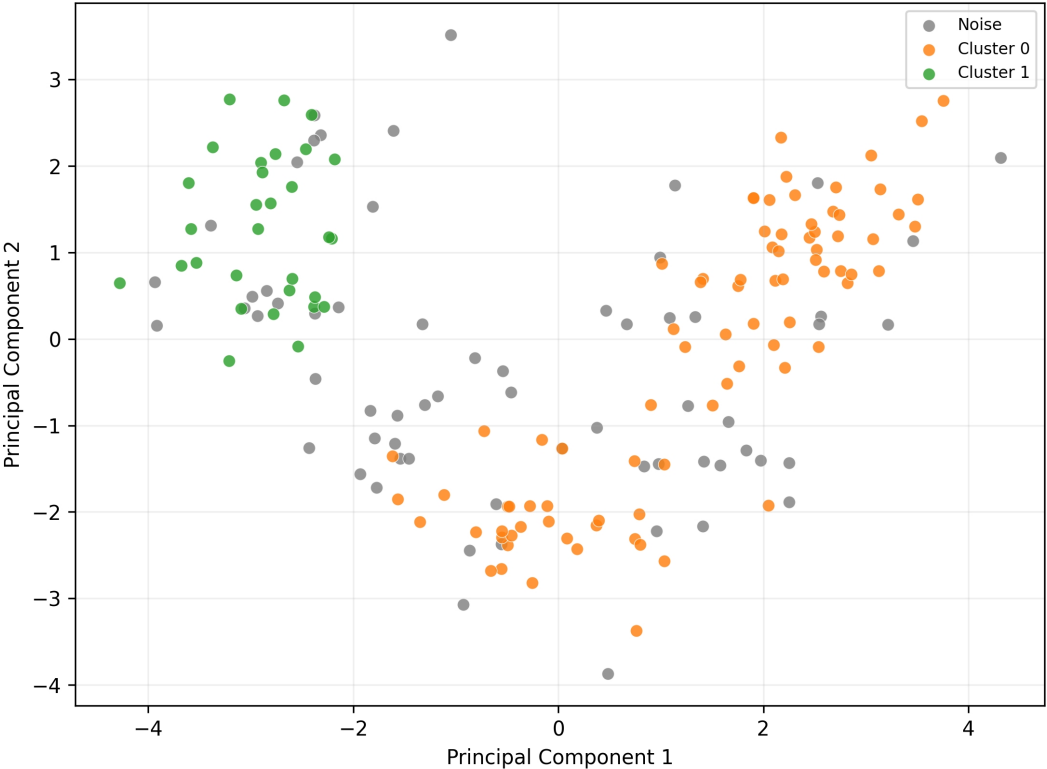


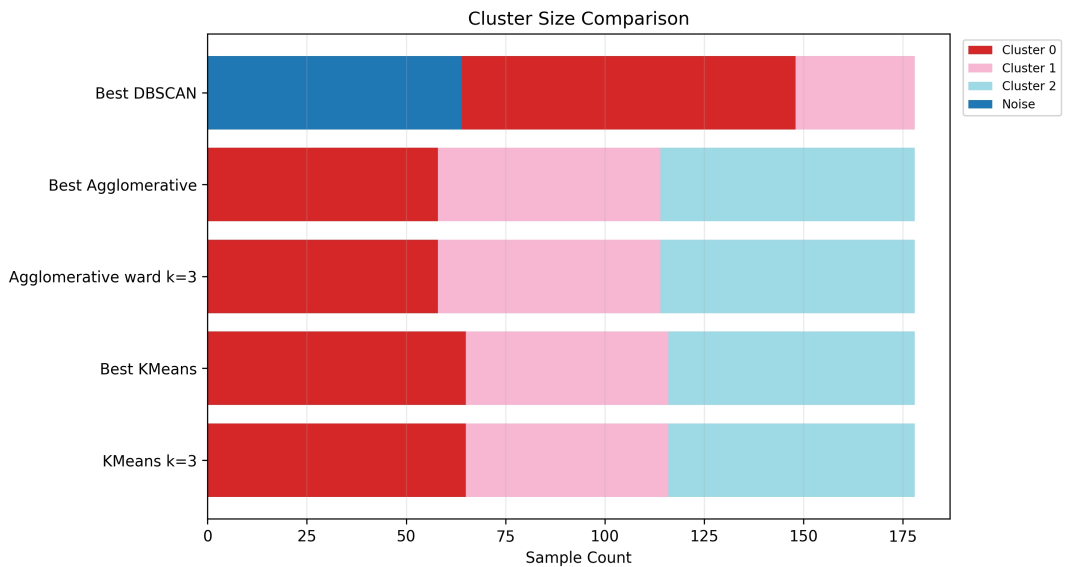
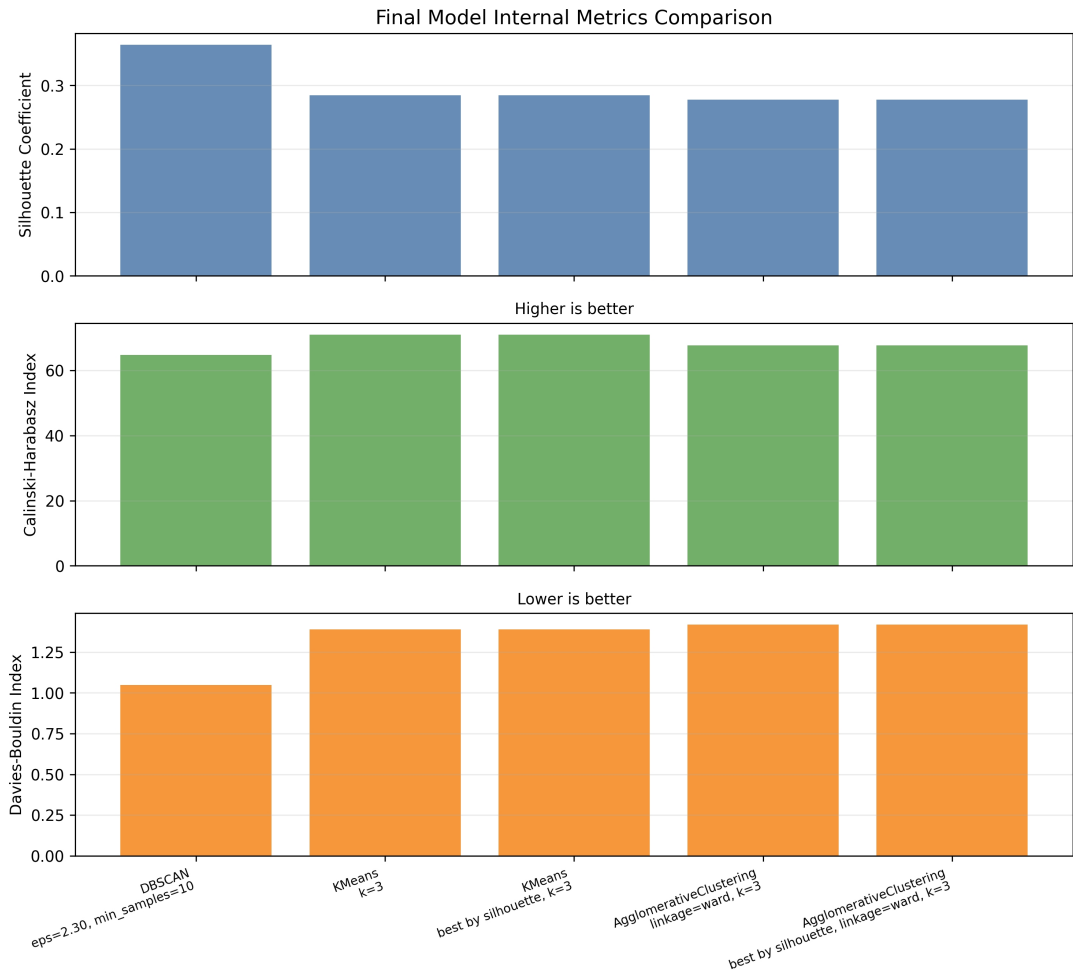


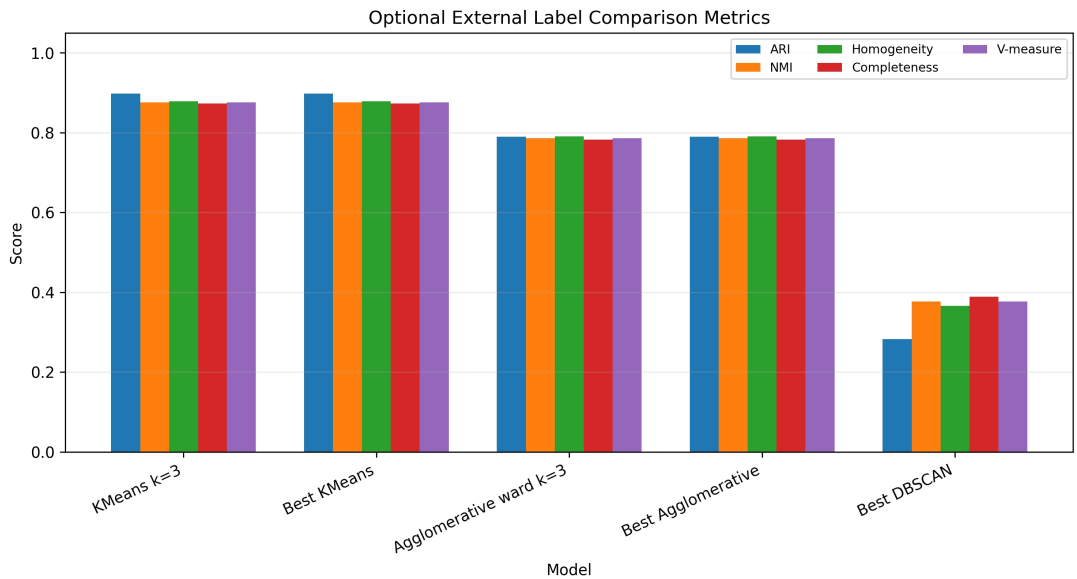
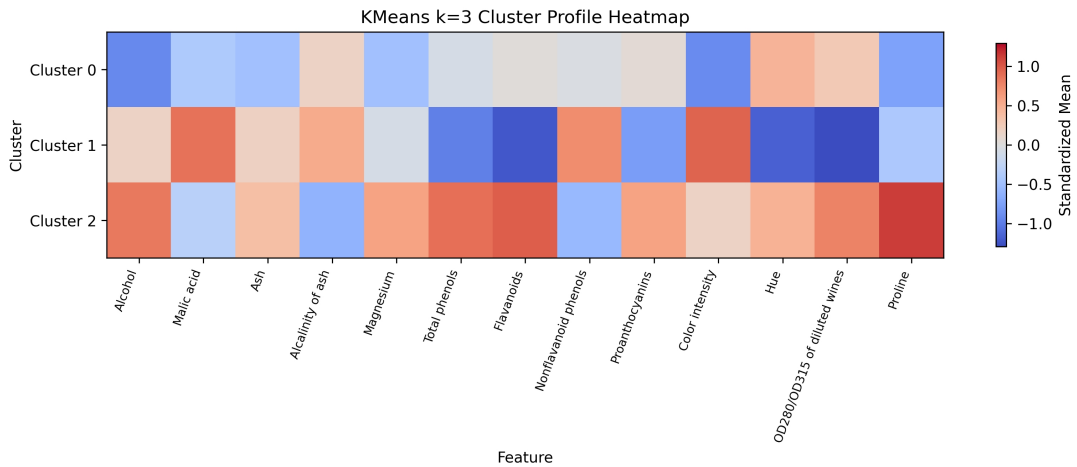




PCA Projection of Best DBSCAN Clusters (eps=2.30, min_samples=10)







From the experimental results, the three clustering methods showed different characteristics on the wine dataset. Before clustering, the original **class** label was removed, and only the **13 wine chemical features** were used as the input features. The dataset contained **178 samples** and **13 features** for clustering, so the experiment followed the requirement of using the wine data as an unsupervised clustering task instead of a classification task. The original class labels were only kept as optional reference information and were not used in model training, parameter selection, or main evaluation.

For **KMeans**, the result of **k = 3** was the most suitable one for the main explanation. When **k = 3**, the model produced three clusters with sample sizes **65, 51, and 62**. Its **Silhouette Coefficient** was **0.2849**, the **Calinski-Harabasz Index** was **70.9400**, and the **Davies-Bouldin Index** was

1.3892. Compared with other k values, $k = 3$ gave a clear and stable three-cluster structure, and every sample was assigned to one cluster. This made **KMeans $k = 3$** easier to interpret in the report.

For **AgglomerativeClustering**, the final hierarchical model used **ward linkage** with $k = 3$. Its **Silhouette Coefficient** was **0.2774**, the **Calinski-Harabasz Index** was **67.6475**, and the **Davies-Bouldin Index** was **1.4186**. These values were close to the KMeans $k = 3$ result, but slightly weaker in the main internal metrics. However, the hierarchical clustering result was still useful because it provided another way to understand the grouping process, especially with the dendrogram figure.

For **DBSCAN**, the best valid setting was **eps = 2.30** and **min_samples = 10**. This setting produced **2 valid clusters** and reached the highest internal metric performance among the final compared models, with **Silhouette Coefficient = 0.3641**, **Calinski-Harabasz Index = 64.7414**, and **Davies-Bouldin Index = 1.0484**. However, its **noise ratio was about 35.96%**, which means that many samples were treated as noise instead of being assigned to normal clusters. Therefore, although DBSCAN had strong internal metric values, it was less convenient for explaining all wine samples clearly.

The cluster profile analysis also supported the use of **KMeans $k = 3$** as the main interpretation model. Cluster 0 was relatively high in **Hue** and **OD280/OD315 of diluted wines**, but relatively low in **Alcohol**, **Color intensity**, and **Proline**. Cluster 1 was relatively high in **Color intensity**, **Malic acid**, and **Nonflavanoid phenols**, but relatively low in **OD280/OD315 of diluted wines**, **Flavanoids**, and **Hue**. Cluster 2 was relatively high in **Proline**, **Flavanoids**, and **Total phenols**, but relatively low in **Alcalinity of ash**, **Nonflavanoid phenols**, and **Malic acid**. These feature differences made the cluster labels more meaningful, because they connected the abstract clustering results with real wine chemical characteristics.

Overall, the result shows that **DBSCAN** achieved the best internal metric scores under the selected parameter setting, but its high noise ratio made it less suitable as the main explanation model. **KMeans $k = 3$** was more stable and easier to interpret because it produced three clear clusters and assigned every sample to a cluster. **AgglomerativeClustering** gave a similar three-cluster result and helped explain the hierarchical grouping process. Therefore, in this report, I treat **DBSCAN** as the best model from the internal metric perspective, but I use **KMeans $k = 3$** as the main model for cluster interpretation. This conclusion also matches the nature of this experiment: the task is **clustering**, so the main evaluation is based on **Silhouette Coefficient**, **Calinski-Harabasz Index**, and **Davies-Bouldin Index**, not classification metrics such as accuracy, precision, recall, or F1-score.

5. Reflections on the Experiment

This experiment gave me a clearer understanding of the difference between **classification** and **clustering**. In the previous experiment, I mainly worked on supervised classification models, so I was used to thinking about a target label, training accuracy, validation accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. However, in this experiment, the task was different. It was a **clustering experiment**, not a classification experiment. This difference seems simple in words, but when I actually wrote the code and prepared the report, I found that it affected almost every part of the workflow.

The first important lesson I learned was about the **wine dataset** itself. The file **wine.data** originally contains a **class** label in the first column, because the dataset was first designed as a classification dataset. At first, it was easy to think that this label might be useful, because it already tells us the original wine type. But Teacher Li Jie reminded us that for clustering, if the dataset has labels, the label should be removed directly before modeling. This point helped me understand the basic rule of unsupervised learning more clearly. In a clustering task, the model should discover groups from the feature patterns, not from the known answer.

Because of this, I paid special attention to the line of logic that saved the original label only as **y_true_optional** and then removed it from the feature matrix. The real clustering input was only the **13 wine chemical features**, such as **Alcohol**, **Malic acid**, **Ash**, **Magnesium**, **Flavanoids**, **Color intensity**, **Hue**, and **Proline**. This step was very important because if I accidentally kept the **class** column in **X**, the clustering result might look good, but the experiment would be wrong in principle. The model would no longer be discovering hidden structure. It would be indirectly using the answer.

The second important lesson was about **evaluation metrics**. In the previous classification experiment, metrics such as **accuracy**, **precision**, **recall**, **F1-score**, **ROC-AUC**, **PR-AUC**, **confusion matrix**, and **classification report** were very useful. But this time, I learned that these classification metrics should not be written as the main evaluation metrics for a clustering experiment. This is one of the points Teacher Li Jie emphasized, and I think it is very important. A metric is not good just because it is familiar. It must match the type of task. Since clustering does not use the original class label during training, the main evaluation should not depend on classification-style prediction correctness.

Therefore, in this experiment, I used **Silhouette Coefficient**, **Calinski-Harabasz Index**, and **Davies-Bouldin Index** as the main clustering evaluation metrics. These metrics helped me evaluate whether the clusters were compact and well separated. This made me realize that clustering evaluation is more indirect than classification evaluation. In classification, I can compare predicted labels with true labels. But in clustering, I need to judge the structure of the data itself. This made the experiment more difficult than I first expected, because there was no single “correct answer” that could simply decide the best model.

Another thing I learned was the importance of **standardization**. Before running the clustering models, I used **StandardScaler** on the 13 wine features. This step was necessary because the features had very different value ranges. For example, **Proline** has a much larger numerical scale than many other chemical features. If I used the raw data directly, the distance calculation might be dominated by large-scale features, and the clustering result could become biased. Through this step, I understood more clearly that preprocessing is not only needed in classification. It is also very important in clustering, especially for distance-based methods.

The three clustering methods also gave me different impressions. **KMeans** was the easiest model to understand and explain. It belongs to **Partition-based Clustering**, and it divides samples into a fixed number of clusters. In my experiment, **KMeans with $k = 3$** gave a clear and stable result, and it assigned every sample to a cluster. Because the wine dataset naturally has a three-class background, KMeans $k = 3$ was very suitable for the main interpretation, although the original class label was not used during training. This helped me understand that prior knowledge can guide explanation, but it should not be used as input for unsupervised modeling.

DBSCAN gave me a different feeling. It belongs to **Density-Based Clustering**, and it can find dense regions while marking some samples as noise. In my result, the best DBSCAN setting had strong internal metric values, but the **noise ratio was about 35.96%**. This means that a relatively large part of the wine samples were treated as noise instead of being assigned to normal clusters. At first, I wanted to simply choose the model with the best metric value. But after checking the noise ratio, I realized that the highest metric score does not always mean the model is the most suitable for explanation. DBSCAN was useful as a density-based comparison model, but for writing a clear report and explaining all samples, **KMeans $k = 3$** was more stable and easier to understand.

AgglomerativeClustering helped me understand **Hierarchical Clustering** better. Different from KMeans, it does not directly start from fixed cluster centers. Instead,

it builds clusters step by step by merging similar samples. The dendrogram figure made this process more visual. I could see that hierarchical clustering is not only about the final cluster labels, but also about the merging process behind them. This made me feel that different clustering models are not just different sklearn function names. They represent different assumptions about how data points should be grouped.

Teacher Li Jie's class helped me a lot in this experiment. What impressed me most was that she did not only ask us to run code. She also emphasized the meaning behind the model and the correct experimental logic. For this experiment, the most important logic was that **wine is a labeled dataset, but clustering must remove the label first**. Another important logic was that **classification metrics should not be mixed into the main evaluation of clustering**. These two reminders helped me avoid writing the report in the wrong direction. I felt that I benefited a lot from her course, because it trained me to think about whether the experiment design is correct, not only whether the program can run.

This experiment also made me more careful about the role of **optional external comparison**. Since the original wine dataset has class labels, I could still compare the final cluster results with the original labels after clustering. Metrics such as **ARI**, **NMI**, **homogeneity**, **completeness**, **v-measure**, and the **class-cluster crosstab** were useful for checking whether the discovered clusters were similar to the original classes. However, I learned that these results should only be treated as optional references. They cannot be used as the main evaluation or as the basis for training and parameter selection. This distinction helped me understand how to use labels carefully in a clustering experiment without breaking the unsupervised setting.

The figures also helped me understand the results more clearly. The **KMeans elbow curve** and **silhouette curve** helped me observe how the number of clusters influenced the result. The **PCA cluster scatter plots** made the clustering structure easier to see. The **DBSCAN heatmaps** showed that DBSCAN was sensitive to **eps** and **min_samples**. The **dendrogram** helped me understand the hierarchical merging process. The **final model metrics comparison** and **cluster profile heatmap** made the final analysis easier to explain. Through these figures, I felt that visualization is especially important in clustering, because clustering results are not as straightforward as classification accuracy.

From the coding side, this experiment also improved my sense of reproducibility. I saved the numerical outputs into the **results** folder and saved the figures into the **figures** folder. I also made sure that **final_cluster_assignments.csv** did not contain

the original **class** label, because the main clustering output should only include sample ids and cluster assignments. The original labels were only saved in optional comparison files. This file organization made the experiment cleaner and easier to check. It also helped me understand that a good experiment should not only produce a result, but also keep the result traceable and explainable.

Compared with the previous classification experiment, this experiment made me think more about the relationship between **data**, **model**, and **evaluation**. In classification, the target label defines the learning goal directly. In clustering, the goal is less direct, so I need to be more careful when interpreting the result. A cluster label like 0, 1, or 2 does not have an original meaning by itself. I need to look at the feature profiles of each cluster to understand what makes one cluster different from another. This made the cluster profile analysis very useful, because it connected the abstract cluster numbers back to real wine chemical features.

Overall, this experiment helped me understand that clustering is not just “classification without labels.” It has its own logic, its own evaluation methods, and its own difficulties. The two points I will remember most are: **first, when doing clustering on a labeled dataset like wine, the class label must be removed before modeling; second, classification metrics such as accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix, and classification report should not be used as the main evaluation metrics for clustering.** These two points may look small, but they decide whether the whole experiment is correct.

In conclusion, Experiment 4 made me more familiar with **Partition-based Clustering**, **Density-Based Clustering**, and **Hierarchical Clustering**. It also helped me understand the practical differences between **KMeans**, **DBSCAN**, and **Agglomerative Clustering**. More importantly, it taught me that a meaningful machine learning experiment should match the model, data, and metrics correctly. For me, the biggest gain from this experiment was not only learning how to run three clustering algorithms, but also learning how to keep the experimental logic clear. This is why I think Teacher Li Jie’s course was very helpful: it made me pay attention to the reasoning behind the workflow, and it helped me build a more careful attitude toward data analysis and machine learning experiments.

Grade:

Reviewed by: -----

Date: -----